90-Day Backend Engineer Learning Plan

Language	C# / ASP.NET Core
Duration	90 Days (May 6 – August 6)
Daily Study	6–8 Hours
Projects	TaskApi · BookstoreApi · eCommerceApi
Goal	Junior-ready backend engineer with real skills

"Understand first. Pseudocode second. Code third. Debug fourth. Repeat."

PHASE OVERVIEW

PHASE 1 — Programming Fundamentals & Thinking (Days 1–14)

Build the mental foundation. Learn to think before you code.

PHASE 2 — Data Structures & Problem Solving (Days 15–28)

Learn the core tools of algorithmic thinking.

PHASE 3 — Backend Fundamentals (Days 29–42)

HTTP, APIs, databases — understand the web backend.

PHASE 4 — Advanced Backend & Full Projects (Days 43–70)

Build real systems. Handle auth, validation, architecture.

PHASE 5 — Polish, Interview Prep & Portfolio (Days 71–90)

Sharpen your edge. Present your best work.

PHASE 1 — Programming Fundamentals & Thinking (Days 1–14)

Build the mental foundation. Learn to think before you code.

Day 1 — How to Think Like a Programmer

■ **Concepts**	• Computational thinking • Problem decomposition • Pseudocode basics • The debugging mindset
■ **Theory & Understanding**	• A program is just a recipe: a precise sequence of instructions. • Break every problem into 3 questions: What do I know? What do I need? What steps connect them? • Pseudocode is English written like code — use it before touching a keyboard. • Analogy: cooking a meal = programming. You don't 'just cook' — you plan ingredients, steps, order.
■ **YouTube Videos**	• 'Think Like a Programmer' — CS Dojo (YouTube) • 'How to start coding' — Fireship (YouTube)
■ **Coding Practice**	• Open a C# console project (dotnet new console). • Write pseudocode (in comments) for: 'Find the largest of 3 numbers'. • Then translate pseudocode to C# code. Run it. • Exercise: 'Is a number even or odd?' — pseudocode first, then code.
■ **Algorithms**	• No LeetCode today. Instead: solve 2 logic puzzles on paper (e.g., FizzBuzz logic by hand). • Write out each step you take — train yourself to slow down and think.
■ **Project Work**	• Setup: Install .NET SDK, VS Code, C# extension. • Create folder: BackendJourney/. Inside: HelloWorld console app. • Print your name, today's date, and your goal to the console.
✍■ **Reflection**	• Write 3 sentences: What did I struggle with today? • What is pseudocode and why does it help? • What's one thing I want to improve tomorrow?

Day 2 — Variables, Types & Control Flow

■ **Concepts**	• Variables and data types • if/else logic • Comparison operators • Console I/O
■ **Theory & Understanding**	• A variable is a named box that holds a value. The type tells C# what kind of value fits in the box. • Types: int (whole numbers), double (decimals), string (text), bool (true/false). • Control flow = the order in which instructions run. if/else lets us choose different paths. • Mental model: a railroad switch — depending on condition, the train goes left or right.
■ **YouTube Videos**	• 'C# Variables and Data Types' — Brackeys (YouTube) • 'If Else in C#' — Programming with Mosh (YouTube)

■ **Coding Practice**	• Write: ask user for age, print 'Minor' if < 18, 'Adult' otherwise. • Write: a simple calculator (add/subtract/multiply/divide) using if/else on operator input. • BEFORE CODING: write pseudocode and trace through the logic manually.
■ **Algorithms**	• LeetCode #1 — Two Sum (understand the problem only today, no code). • Write on paper: what are the inputs? What is the output? What are edge cases?
■ **Project Work**	• Add to HelloWorld: ask the user their name, greet them personally. • Use if/else to print a different message based on the hour of day (Morning/Afternoon/Evening).
✍■ **Reflection**	• What is the difference between int and double? • Why do we write pseudocode before code? • What confused me today and how did I resolve it?

Day 3	Loops & Iteration
■ Concepts	• for loops • while loops • Loop control: break/continue • Iteration patterns
■ Theory & Understanding	• A loop lets you repeat instructions without copy-pasting. It's automation. • for loop = when you know exactly how many times to repeat. • while loop = when you repeat until some condition becomes false. • Analogy: a for loop is like doing 10 push-ups. A while loop is like eating until you're full. • DANGER: always make sure the loop has an exit condition — infinite loops crash your program.
■ YouTube Videos	• 'C# Loops' — Brackeys (YouTube) • 'For vs While Loop explained' — CS Dojo (YouTube)
■ Coding Practice	• Print numbers 1 to 100 using a for loop. • Print only even numbers from 1 to 50. • Use a while loop: keep asking user for a number until they enter 0. • FizzBuzz: for 1-30, print Fizz if divisible by 3, Buzz by 5, FizzBuzz by both, else number.
■ Algorithms	• LeetCode #1 — Two Sum: now write a brute-force O(n^2) solution using nested loops. • Trace through your solution on paper with a small input before running it.
■ Project Work	• Guess the Game v1: generate a random number 1-100, let the user guess, tell them higher/lower. Keep going until correct. Print how many guesses it took.
✍■ Reflection	• What is the difference between for and while? • What happens if you forget to increment a counter in a while loop? • Did I write pseudocode before coding today?

Day 4	Methods / Functions — Reusable Logic
■ Concepts	• Defining methods • Parameters and return values • Method signatures • Why we use methods
■ Theory & Understanding	• A method is a named block of code that does one specific job. You call it whenever you need that job done. • Why? DRY — Don't Repeat Yourself. Write once, use many times. • Parameters = inputs to the method. Return value = output from it. • Analogy: a method is like a vending machine. You put in money (parameters), press a button (call it), get a snack (return value). • Rule of thumb: a method should do ONE thing and be short enough to read at a glance.
■ YouTube Videos	• 'C# Methods' — Programming with Mosh (YouTube)
■ Coding Practice	• Write a method: int Add(int a, int b) — returns sum. • Write: bool IsEven(int n) — returns true if even. • Write: string Greet(string name) — returns 'Hello, {name}!' • Refactor your FizzBuzz from Day 3 into a method: string FizzBuzz(int n).

■ **Algorithms**	• LeetCode #412 — FizzBuzz (easy). Use your method from today's coding. • LeetCode #771 — Jewels and Stones (easy). Trace on paper first.
■ **Project Work**	• Refactor Guess the Game: move 'check guess' logic into a method CheckGuess(int secret, int guess) → returns 'higher', 'lower', or 'correct'.
✍■ **Reflection**	• What does 'return' do in a method? • Why is a method that does ONE thing better than one that does many? • What was the hardest part of refactoring today?

Day 5	Arrays & Lists
■ Concepts	• Arrays (fixed size) • List (dynamic) • Indexing and iteration • Common operations: add, remove, search
■ Theory & Understanding	• An array is a row of numbered boxes, all holding the same type of value. • Arrays have a fixed size — decided when created. Lists grow and shrink dynamically. • Indexing starts at 0. The first element is [0], not [1]. This trips up beginners constantly. • Analogy: an array is a parking lot with fixed spots. A list is a flexible queue. • When to use array: size is known and fixed. When to use List: size changes over time.
■ YouTube Videos	• 'C# Arrays and Lists' — Brackeys (YouTube)
■ Coding Practice	• Create an int[] of 5 numbers. Print each using a for loop. • Create a List of 5 names. Add 2 more, remove 1, print all. • Write a method: int FindMax(int[] nums) — returns the largest number. • Write: bool Contains(List items, string target) — returns true if target is in list.
■ Algorithms	• LeetCode #26 — Remove Duplicates from Sorted Array. • LeetCode #1 — Two Sum: now solve it using a Dictionary (HashMap) for O(n) — just understand the approach.
■ Project Work	• Extend Guess the Game: store all previous guesses in a List. At the end, print the full guess history.
✍■ Reflection	• What is the difference between an array and a list? • Why does indexing start at 0? • What is an 'IndexOutOfRange' error and when does it happen?

Day 6	Strings — Working with Text
■ Concepts	• String methods • String formatting • StringBuilder • Immutability of strings
■ Theory & Understanding	• Strings are sequences of characters. In C#, strings are immutable — you can't change them, only create new ones. • This matters for performance: never use string += inside a big loop. Use StringBuilder instead. • Key methods: Length, ToUpper(), ToLower(), Trim(), Split(), Replace(), Contains(), StartsWith(), Substring(). • String interpolation: $"Hello, {name}!" — cleaner than concatenation.
■ YouTube Videos	• 'Working with Strings in C#' — Programming with Mosh (YouTube)
■ Coding Practice	• Write: bool IsPalindrome(string s) — returns true if s reads the same forwards and backwards. • Write: int CountVowels(string s) — count a, e, i, o, u. • Write: string ReverseWords(string sentence) — reverse word order. • Trace each function on paper with a sample input before running.
■ Algorithms	• LeetCode #125 — Valid Palindrome (easy). • LeetCode #344 — Reverse String (easy).

■ **Project Work**	• Build a console-based 'Name Formatter': ask for first/last name, print formatted versions (UPPERCASE, title case, reversed, initials).
✍■ **Reflection**	• What does 'immutable' mean and why does it matter for strings? • When should you use StringBuilder vs string? • What is the most useful string method I learned today?

Day 7	Review & Consolidation Day
■ Concepts	• Review all concepts from Days 1–6 • Fill gaps • Reinforce weak areas
■ Theory & Understanding	• Today is NOT a rest day. It is a catch-up and strengthening day. • Go back to any concept that felt unclear and re-read it slowly. • The goal is: no concept from Days 1-6 should feel shaky after today.
■ YouTube Videos	• Re-watch one video from earlier this week that confused you.
■ Coding Practice	• Complete any exercises you skipped or half-finished. • Write from scratch (no looking): FizzBuzz, FindMax, IsPalindrome. • Time yourself: how fast can you write a correct, working solution?
■ Algorithms	• LeetCode #9 — Palindrome Number. • LeetCode #242 — Valid Anagram. • For each: write full solution in pseudocode first, then code.
■ Project Work	• Polish Guess the Game: add error handling for non-numeric input, add a 'play again' option.
✍■ Reflection	• What 3 concepts am I most confident with? • What 1 concept do I still feel weak on? • What is my plan to strengthen that weak area?

Day 8	Classes & Objects — Object-Oriented Thinking
■ Concepts	• Classes and objects • Fields and properties • Constructors • Encapsulation basics
■ Theory & Understanding	• A class is a blueprint. An object is a thing built from that blueprint. • Analogy: a class is the architectural plan for a house. Objects are actual houses built from it. • Fields store state. Properties expose that state with controlled access. • Encapsulation: hide internal details. Expose only what is needed. • Constructor: the method called when you create a new object (new MyClass()).
■ YouTube Videos	• 'C# Classes and Objects' — Programming with Mosh (YouTube)
■ Coding Practice	• Create a class: Person with Name (string), Age (int), and a Greet() method. • Create a class: Rectangle with Width, Height, and methods Area() and Perimeter(). • Instantiate 3 Person objects and store them in a List. Print all greetings.
■ Algorithms	• LeetCode #383 — Ransom Note (easy). • LeetCode #217 — Contains Duplicate (easy).
■ Project Work	• Start: Student Grade Tracker console app. • Create class: Student { Name, List Grades }. • Add method: double Average(). Add method: string LetterGrade(). • In main: create 3 students, print name, average, and letter grade.

✍■ **Reflection**	• What is the difference between a class and an object? • What is encapsulation and why is it useful? • What would happen if all fields were public with no validation?

Day 9	Inheritance & Polymorphism
■ Concepts	• Inheritance (base/derived classes) • Method overriding • Polymorphism • Abstract classes
■ Theory & Understanding	• Inheritance: a child class inherits all behaviour from its parent. • Analogy: Dog inherits from Animal. A dog IS-AN animal but adds its own behaviour. • Polymorphism: different objects respond differently to the same method call. • Override: the child rewrites the parent's method with its own version. • Abstract class: a parent class that can never be instantiated — exists only to be inherited. • When to use: when multiple types share common behaviour but also have unique behaviour.
■ YouTube Videos	• 'C# Inheritance' — Programming with Mosh (YouTube)
■ Coding Practice	• Create base class: Shape with abstract method Area(). • Create derived classes: Circle, Rectangle, Triangle — each implements Area(). • Create a List, add all three types, print each area polymorphically.
■ Algorithms	• LeetCode #206 — Reverse Linked List (understand the concept, trace on paper). • LeetCode #70 — Climbing Stairs (easy dynamic programming — just read and understand).
■ Project Work	• Grade Tracker: add a Teacher class (inherits from Person from Day 8). • Teacher has a List and a method PrintClassReport().
✍■ Reflection	• What is the 'is-a' relationship in inheritance? • When should you use inheritance vs just having separate classes? • What does 'override' mean and why is it needed?

Day 10	Interfaces & Abstraction
■ Concepts	• Interfaces • Why interfaces over inheritance • Dependency Inversion • SOLID preview
■ Theory & Understanding	• An interface is a contract. Any class that implements it MUST provide those methods. • Analogy: a power outlet interface. Any plug that fits the shape gets power — regardless of what the appliance is. • Interfaces enable swapping implementations without changing calling code. • This is core to professional C# — ASP.NET Core is built on interfaces everywhere. • Difference: inheritance = 'is-a'. Interface = 'can-do'.
■ YouTube Videos	• 'C# Interfaces' — Programming with Mosh (YouTube)
■ Coding Practice	• Define interface: IShape { double Area(); double Perimeter(); } • Implement it in Circle, Rectangle, Triangle. • Write a method: void PrintShapeInfo(IShape shape) — works for any shape. • Try: what happens if a class only partially implements an interface?
■ Algorithms	• LeetCode #20 — Valid Parentheses (stack concept — read theory first). • LeetCode #155 — Min Stack (easy).

■ **Project Work**	• Refactor Grade Tracker to use an interface: IGradable { double Average(); string LetterGrade(); } • Both Student and a new class CourseSection implement IGradable.
✍■ **Reflection**	• What is the difference between an interface and an abstract class? • Why would you use an interface in a real project? • What is the SOLID principle 'D' (Dependency Inversion) in simple terms?

Day 11	Error Handling & Debugging
■ Concepts	• try/catch/finally • Exception types • Throwing exceptions • Debugging with breakpoints
■ Theory & Understanding	• Errors come in two types: compile errors (caught before running) and runtime exceptions (crashes during run). • try/catch lets your program gracefully handle unexpected situations. • Never use catch(Exception e) {} with an empty body — that buries bugs forever. • Good error handling: catch specific exceptions, log what happened, give the user useful information. • Debugging: use breakpoints to pause execution and inspect variable values line by line.
■ YouTube Videos	• 'C# Exception Handling' — Programming with Mosh (YouTube)
■ Coding Practice	• Write a method: int Divide(int a, int b) — catch DivideByZeroException. • Write a method: int ParseAge(string input) — catch FormatException. • Deliberately introduce a bug in old code. Use VS Code debugger to find it with breakpoints.
■ Algorithms	• LeetCode #14 — Longest Common Prefix (easy). • LeetCode #13 — Roman to Integer (easy) — trace manually first.
■ Project Work	• Add error handling to Grade Tracker: validate that grades are between 0-100. Throw an ArgumentException if not.
✍■ Reflection	• What is the difference between a compile error and a runtime exception? • When should you throw a custom exception vs return an error code? • What was the bug you introduced today and how did you find it?

Day 12	LINQ & Functional-Style Collections
■ Concepts	• LINQ basics • Where, Select, OrderBy, First, Count, Any, All • Lambda expressions • Chaining queries
■ Theory & Understanding	• LINQ (Language INtegrated Query) lets you query collections in a declarative style. • Instead of: write a loop to filter → LINQ: .Where(x => x.Age > 18). • Lambdas: short anonymous methods. x => x * 2 means 'take x, return x times 2'. • LINQ is deferred — the query doesn't run until you enumerate the result (e.g., with ToList()). • Real-world use: filtering database results, transforming data, sorting lists — everywhere in C#.
■ YouTube Videos	• 'C# LINQ Tutorial' — IAmTimCorey (YouTube)
■ Coding Practice	• Given a List, use LINQ to: filter evens, find max, sort descending, sum all. • Given a List (from project), use LINQ to: find top student, get all with avg > 80, sort by name.
■ Algorithms	• LeetCode #350 — Intersection of Two Arrays II. • LeetCode #169 — Majority Element.
■ Project Work	• Grade Tracker: add LINQ-powered reports — top 3 students, class average, students below 60.

✍■ **Reflection**	• What does 'deferred execution' mean in LINQ? • When is LINQ cleaner than a for loop? When is a loop better? • Write from memory: how to filter a list of ints to only include those > 10.

Day 13	File I/O & JSON Basics
■ Concepts	• Reading/writing files • System.IO namespace • JSON serialization • Newtonsoft.Json / System.Text.Json
■ Theory & Understanding	• Real applications persist data — they save and load state between runs. • File I/O: read text files line by line, write output to files. • JSON (JavaScript Object Notation): the universal format for data interchange. Humans can read it. Machines love it. • Serialization = converting a C# object to JSON text. Deserialization = JSON text → C# object. • Every backend API you'll build will send and receive JSON. Master this now.
■ YouTube Videos	• 'JSON in C#' — IAmTimCorey (YouTube)
■ Coding Practice	• Write and read a text file containing a list of names. • Serialize a Student object to JSON. Save it to file. Load it back and print. • Serialize a List to JSON and back.
■ Algorithms	• LeetCode #268 — Missing Number. • LeetCode #283 — Move Zeroes.
■ Project Work	• Grade Tracker: add Save() and Load() methods. Students are stored in a JSON file. When app starts, load existing data.
✍■ Reflection	• What is JSON and why is it the standard for APIs? • What is the difference between serialization and deserialization? • What file-handling error could break the app if not handled?

Day 14	Phase 1 Review & Mini Project Completion
■ Concepts	• Full review of Phase 1 • Code quality and clean code • Refactoring
■ Theory & Understanding	• Clean code: code that is easy to READ, not just correct. • Naming: variables and methods should say what they do. No 'x', 'temp', 'doStuff'. • Short methods: if a method is > 20 lines, it's probably doing too much. • No magic numbers: const int MAX_STUDENTS = 30; not hardcoded 30 everywhere.
■ YouTube Videos	• 'Clean Code in C#' — Nick Chapsas (YouTube)
■ Coding Practice	• Review all code written in Phase 1. Refactor names, extract long methods, remove duplication. • Write 3 methods from scratch without notes: FindMax, IsPalindrome, CountVowels.
■ Algorithms	• LeetCode #1 — Two Sum (solve it again, clean solution, O(n)). • LeetCode #121 — Best Time to Buy and Sell Stock.
■ Project Work	• Complete and polish Grade Tracker: full CRUD (add/remove students), LINQ reports, file persistence, error handling, clean code. • Write a README.md explaining what the project does.

✍■ **Reflection**	• What are the 3 most important things I learned in Phase 1? • What coding habits do I need to build? • Am I ready for Phase 2? What do I still need to review?

PHASE 2 — Data Structures & Problem Solving (Days 15–28)

Learn the core tools of algorithmic thinking.

Day 15 | Big-O Notation & Algorithm Analysis

■ **Concepts**	• Time complexity • Space complexity • O(1), O(n), O(n²), O(log n) • Why complexity matters
■ **Theory & Understanding**	• Big-O describes HOW FAST a solution's cost grows as the input grows. • O(1) = constant — no matter how big the input, same cost. Example: array[i]. • O(n) = linear — cost grows proportionally with input. Example: loop through all elements. • O(n²) = quadratic — nested loops. Fine for small n, disastrous for large n. • O(log n) = logarithmic — halves the problem each step. Binary search. • Rule: before submitting any solution, state its Big-O.
■ **YouTube Videos**	• 'Big O Notation' — CS Dojo (YouTube)
■ **Coding Practice**	• Annotate all your Phase 1 solutions with their time/space complexity. • Write both O(n²) and O(n) solutions for Two Sum and compare.
■ **Algorithms**	• LeetCode #217 — Contains Duplicate (solve in O(n) with HashSet). • LeetCode #53 — Maximum Subarray (understand brute force, then Kadane's).
■ **Project Work**	• No new feature. Read your Grade Tracker code and annotate each method with its Big-O.
✍■ **Reflection**	• What is the difference between time and space complexity? • Why is O(n log n) better than O(n²) for large inputs? • What is Kadane's algorithm doing, in plain English?

Day 16 | Arrays — Deep Dive & Patterns

■ **Concepts**	• Two-pointer technique • Sliding window preview • Prefix sums • Array manipulation patterns
■ **Theory & Understanding**	• Two pointers: use two indices (left and right) that move toward each other or in the same direction. • Use case: sorted arrays, palindrome checks, pair-sum problems. • Prefix sum: precompute cumulative sums so range-sum queries become O(1) instead of O(n). • Sliding window: maintain a window of elements that slides across the array — avoids re-computation.
■ **YouTube Videos**	• 'Two Pointer Technique' — NeetCode (YouTube)
■ **Coding Practice**	• Implement two-pointer: find if any pair in sorted array sums to target. • Implement prefix sum: precompute prefix, then answer range-sum queries in O(1).
■ **Algorithms**	• LeetCode #167 — Two Sum II (sorted array, two pointers). • LeetCode #11 — Container With Most Water. • LeetCode #303 — Range Sum Query (prefix sum).

■ **Project Work**	• Start: Inventory Manager console app (will grow through Phase 2). Create Item class, List store.
✍■ **Reflection**	• Why does two-pointer work on SORTED arrays? • What makes prefix sum powerful? • Which problem today was hardest and why?

Day 17	Dictionaries / Hash Maps
■ Concepts	• Dictionary • Hashing concept • HashSet • Counting and grouping patterns
■ Theory & Understanding	• A dictionary maps a key to a value. Looking up by key is O(1) on average. • Why O(1)? Hashing: the key is converted to an index via a hash function — direct access, no searching. • HashSet: like a dictionary with only keys — used for fast existence checks. • Common pattern: frequency counting — count how many times each element appears. • Analogy: a dictionary is like a library catalogue. You don't search every book — you look up the index.
■ YouTube Videos	• 'Hash Tables Explained' — CS Dojo (YouTube)
■ Coding Practice	• Word frequency counter: given a string, count occurrences of each word using Dictionary. • Group students by letter grade using GroupBy / Dictionary.
■ Algorithms	• LeetCode #1 — Two Sum (Dictionary solution, fully understand it). • LeetCode #49 — Group Anagrams. • LeetCode #347 — Top K Frequent Elements.
■ Project Work	• Inventory Manager: add lookup by item name (O(1) with Dictionary), track stock counts.
✍■ Reflection	• Why is Dictionary lookup O(1) on average? • What is a hash collision and how does C# handle it? • When would you use HashSet instead of List?

Day 18	Stacks & Queues
■ Concepts	• Stack (LIFO) • Queue (FIFO) • Stack and Queue in C# • Real-world applications
■ Theory & Understanding	• Stack: Last In, First Out. Like a stack of plates — you always take from the top. • Queue: First In, First Out. Like a supermarket queue — first person in is first served. • Stack applications: undo/redo, expression parsing, call stack, DFS traversal. • Queue applications: BFS traversal, task scheduling, message queues. • Don't implement from scratch yet — use C# built-ins. Understand behaviour first.
■ YouTube Videos	• 'Stacks and Queues' — CS Dojo (YouTube)
■ Coding Practice	• Use Stack to check balanced parentheses: (), {}, []. • Use Queue to simulate a print queue (add jobs, process in order).
■ Algorithms	• LeetCode #20 — Valid Parentheses (stack solution). • LeetCode #232 — Implement Queue using Stacks. • LeetCode #739 — Daily Temperatures (monotonic stack — read and understand).
■ Project Work	• Inventory Manager: add a transaction history using Queue (FIFO order of stock changes).

✍■ **Reflection**	• What is the difference between Stack and Queue behaviour? • Why is a stack used for balanced parentheses checking? • What is a monotonic stack and when is it useful?

Day 19	Linked Lists — Concept & Implementation
■ Concepts	• Singly linked list • Node structure • Traversal, insertion, deletion • vs Array comparison
■ Theory & Understanding	• A linked list is a chain of nodes. Each node holds a value AND a pointer to the next node. • Unlike an array, elements are NOT stored contiguously in memory. • Arrays: O(1) random access, O(n) insert/delete. Linked lists: O(n) access, O(1) insert/delete at front. • LeetCode uses linked lists heavily — understand the pattern, not just memorise. • Analogy: a scavenger hunt — each clue tells you where the next clue is.
■ YouTube Videos	• 'Linked Lists' — CS Dojo (YouTube)
■ Coding Practice	• Implement: class Node { int Value; Node Next; } • Implement a simple LinkedList class: Add, Remove, Print, Contains. • Write: reverse a linked list iteratively (in-place).
■ Algorithms	• LeetCode #206 — Reverse Linked List. • LeetCode #21 — Merge Two Sorted Lists. • LeetCode #141 — Linked List Cycle (fast/slow pointer).
■ Project Work	• No new project feature. Focus on coding exercises — linked lists are interview staples.
✍■ Reflection	• What is a null pointer and why does it matter in linked lists? • Why can't you access a linked list element by index in O(1)? • Explain fast/slow pointer technique in plain English.

Day 20	Recursion — Think in Self-Reference
■ Concepts	• Base case and recursive case • Call stack visualization • Recursion vs iteration • Common recursive patterns
■ Theory & Understanding	• Recursion: a method calls itself with a smaller version of the problem. • ALWAYS needs a base case — the condition that stops it. Without it: infinite recursion = stack overflow. • Mental model: Russian dolls. Open the outer doll — inside is a smaller version of the same problem. • Every recursive solution can be done iteratively (but some are much cleaner recursively). • Recursion is foundational for trees, graphs, and dynamic programming coming up.
■ YouTube Videos	• 'Recursion for Beginners' — CS Dojo (YouTube)
■ Coding Practice	• Factorial(n) — iteratively AND recursively. • Fibonacci(n) — recursive (understand why it's slow — O(2^n)). • Power(base, exp) — recursive: base^exp = base * Power(base, exp-1). • Flatten a nested list recursively.

■ **Algorithms**	• LeetCode #509 — Fibonacci Number. • LeetCode #206 — Reverse Linked List (recursive solution this time). • LeetCode #112 — Path Sum (binary tree — trace structure on paper first).
■ **Project Work**	• Inventory Manager: add recursive search — find an item by partial name in nested categories.
✍■ **Reflection**	• What are the two parts every recursive function must have? • What is a stack overflow error in the context of recursion? • When is recursion better than iteration?

Day 21	Review Day — Phase 2 Midpoint
■ Concepts	• Big-O review • Patterns: two-pointer, hashing, stack, recursion
■ Theory & Understanding	• No new concepts today. The goal is to solidify what you have. • Strong foundations at this point means faster progress in Weeks 5–9.
■ YouTube Videos	• Optional: revisit one confusing video from this week.
■ Coding Practice	• Write from scratch: IsPalindrome (two pointer), word frequency counter (hash map), balanced parentheses (stack). • Without notes — time yourself.
■ Algorithms	• LeetCode #75 — Sort Colors (two pointers). • LeetCode #387 — First Unique Character in a String.
■ Project Work	• Polish Inventory Manager: clean code, add file persistence (JSON save/load).
✍■ Reflection	• Which data structure am I weakest on? • Can I state Big-O for every solution I wrote this week? • What is my plan to strengthen the weak areas before Phase 3?

Day 22	Trees — Binary Trees & BSTs
■ Concepts	• Tree structure and terminology • Binary tree • Binary Search Tree (BST) • Tree traversals: inorder, preorder, postorder
■ Theory & Understanding	• A tree is a hierarchical data structure. Each node has a value and up to 2 children (binary tree). • BST property: left child < parent < right child. Enables O(log n) search. • Inorder traversal of a BST gives elements in sorted order. • Preorder = root first. Inorder = left first. Postorder = right first. • Analogy: a family tree. Each person is a node. Lines to children are edges.
■ YouTube Videos	• 'Binary Trees' — CS Dojo (YouTube)
■ Coding Practice	• Implement: TreeNode class { int Val; TreeNode Left, Right; } • Write: inorder, preorder, postorder traversal (recursive). • Write: int Height(TreeNode root) — returns tree height.
■ Algorithms	• LeetCode #104 — Maximum Depth of Binary Tree. • LeetCode #226 — Invert Binary Tree. • LeetCode #543 — Diameter of Binary Tree.
■ Project Work	• No project work. Tree problems require full focus.
✍■ Reflection	• What is the BST property and why does it enable fast search? • What is the height of a binary tree? • Draw a tree on paper and manually trace an inorder traversal.

Day 23	Graph Basics — BFS & DFS
■ Concepts	• Graph representation (adjacency list) • Depth-First Search (DFS) • Breadth-First Search (BFS) • When to use each
■ Theory & Understanding	• A graph is nodes (vertices) connected by edges. Unlike trees, cycles are allowed. • Adjacency list: for each node, store a list of its neighbours. Most common representation. • DFS: go deep first. Uses a stack (or recursion). • BFS: explore level by level. Uses a queue. Finds shortest paths in unweighted graphs. • When: BFS for shortest path, DFS for exhaustive exploration (backtracking, cycle detection).
■ YouTube Videos	• 'Graph Algorithms for Beginners' — freeCodeCamp (YouTube)
■ Coding Practice	• Build a graph using Dictionary>. • Implement DFS iteratively (with stack) and recursively. • Implement BFS using Queue.
■ Algorithms	• LeetCode #200 — Number of Islands (DFS). • LeetCode #695 — Max Area of Island. • LeetCode #733 — Flood Fill.
■ Project Work	• No project work. Full focus on graphs — this is heavy material.
✍■ Reflection	• What is the key difference between DFS and BFS? • Why does BFS find shortest paths in unweighted graphs? • What does 'visited' tracking prevent in graph traversal?

Day 24	Dynamic Programming — Introduction
■ Concepts	• Overlapping subproblems • Memoization (top-down) • Tabulation (bottom-up) • Classic DP: Fibonacci, Coin Change
■ Theory & Understanding	• DP is for problems that have OVERLAPPING SUBPROBLEMS — the same sub-calculation repeated many times. • Memoization: cache results of recursive calls so you don't repeat work. • Tabulation: build a table bottom-up, filling in base cases first. • Fibonacci with recursion is O(2^n). With DP it becomes O(n). • DP is hard — don't expect to master it in one day. Today: recognize the PATTERN.
■ YouTube Videos	• 'Dynamic Programming — Learn to Solve Algorithmic Problems' — freeCodeCamp (YouTube)
■ Coding Practice	• Fibonacci: naive recursion → add memoization → rewrite as tabulation. • Climbing Stairs: how many ways to climb n stairs taking 1 or 2 steps?
■ Algorithms	• LeetCode #70 — Climbing Stairs. • LeetCode #198 — House Robber. • LeetCode #509 — Fibonacci (memoized).
■ Project Work	• No project work. DP requires dedicated focus.

✍■ **Reflection**	• What is the difference between memoization and tabulation? • What makes a problem suitable for dynamic programming? • Explain Climbing Stairs solution in plain English.

Day 25	Sorting Algorithms
■ Concepts	• Bubble sort, Selection sort (understand, don't use) • Merge sort, Quick sort (understand) • C# built-in sort • When sorting matters
■ Theory & Understanding	• Bubble/Selection sort: O(n²) — simple but slow. Understand the concept, never use in production. • Merge sort: O(n log n) — divide and conquer. Stable sort. Used in practice. • Quick sort: O(n log n) average — pivot partitioning. C# Array.Sort uses a variant. • In interviews: you rarely implement sorts. But you MUST know their complexities. • Key insight: sorting often transforms a hard problem into an easy one. Always consider 'what if I sort this first?'
■ YouTube Videos	• 'Sorting Algorithms' — Fireship (YouTube, 10 mins)
■ Coding Practice	• Implement bubble sort to understand the mechanics. • Implement merge sort (practice divide and conquer thinking). • Use Array.Sort with a custom Comparer on a list of students.
■ Algorithms	• LeetCode #912 — Sort an Array (implement merge sort). • LeetCode #75 — Sort Colors (Dutch National Flag — 3-way partition).
■ Project Work	• Inventory Manager: sort items by price, by name, by stock level using LINQ and custom comparers.
✍■ Reflection	• Why is merge sort O(n log n) and bubble sort O(n²)? • What does 'stable sort' mean? • When should you sort an array to make a problem easier?

Day 26	Binary Search
■ Concepts	• Binary search algorithm • Search on sorted arrays • Binary search on answer • Template pattern
■ Theory & Understanding	• Binary search: repeatedly halve the search space by comparing the midpoint. • Precondition: the array MUST be sorted. Otherwise: undefined behaviour. • O(log n) — each step eliminates half the remaining options. • Pattern: find the leftmost valid answer using lo/hi/mid pointers. • Real use: search in sorted arrays, database index lookups, parametric search.
■ YouTube Videos	• 'Binary Search' — NeetCode (YouTube)
■ Coding Practice	• Implement binary search from scratch. No looking at references. • Write both iterative and recursive versions. • Trace through [1,3,5,7,9,11] searching for 7 — write each step.
■ Algorithms	• LeetCode #704 — Binary Search. • LeetCode #35 — Search Insert Position. • LeetCode #153 — Find Minimum in Rotated Sorted Array.

■ **Project Work**	• Inventory Manager: implement binary search on sorted-by-name item list.
✍■ **Reflection**	• Why must an array be sorted for binary search to work? • What is off-by-one error in binary search and how do you avoid it? • What does 'lo = mid + 1' vs 'lo = mid' mean in terms of convergence?

Day 27	Sliding Window Pattern
■ Concepts	• Fixed-size sliding window • Variable-size sliding window • When to use it • Avoiding O(n²)
■ Theory & Understanding	• Sliding window: maintain a contiguous sub-array or sub-string that 'slides' through the input. • Avoids re-computing from scratch for each position — reuse previous computation. • Fixed window: window size is given (e.g., max sum of k consecutive elements). • Variable window: expand right, shrink left until condition is met. • Classic use: longest substring without repeating characters, maximum sum subarray of size k.
■ YouTube Videos	• 'Sliding Window Technique' — NeetCode (YouTube)
■ Coding Practice	• Find max sum of any k consecutive elements in array. • Find smallest subarray with sum ≥ target.
■ Algorithms	• LeetCode #643 — Maximum Average Subarray I. • LeetCode #3 — Longest Substring Without Repeating Characters. • LeetCode #209 — Minimum Size Subarray Sum.
■ Project Work	• No project today. Sliding window is a critical pattern — full focus.
✍■ Reflection	• Why is sliding window O(n) and not O(n²)? • What is the difference between fixed and variable window? • What data structure helps track elements in the current window?

Day 28	Phase 2 Review & Inventory Manager Completion
■ Concepts	• Full review of data structures and algorithms • Refactor and complete project
■ Theory & Understanding	• Consolidation day. Go through every data structure and algorithm from this phase. • Make a cheat sheet (one page): each data structure, when to use it, its Big-O for common operations.
■ YouTube Videos	• No new video. Review notes.
■ Coding Practice	• From memory: implement binary search, reverse linked list, BFS, memoized fibonacci. • Annotate every solution with its time/space complexity.
■ Algorithms	• LeetCode #238 — Product of Array Except Self. • LeetCode #424 — Longest Repeating Character Replacement.
■ Project Work	• Complete Inventory Manager: CRUD, search (binary search), sort (custom), file persistence, error handling. • Write a README. Push to GitHub.
✍■ Reflection	• What is the most important algorithmic pattern I learned in Phase 2? • Which data structure do I still feel uncertain about? • Am I ready to start building APIs?

PHASE 3 — Backend Fundamentals (Days 29–42)

HTTP, APIs, databases — understand the web backend.

Day 29 | How the Web Works — HTTP & the Backend

■ **Concepts**	• Client-server model • HTTP methods: GET, POST, PUT, DELETE • Status codes • Request/Response cycle
■ **Theory & Understanding**	• A backend is a server that receives requests, processes them, and sends responses. • HTTP is the language browsers and servers use to talk. Every web request uses it. • Methods: GET = read, POST = create, PUT = update, DELETE = delete. • Status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error. • Analogy: a restaurant. Client = customer. Server = waiter. Backend logic = kitchen. Response = your meal.
■ **YouTube Videos**	• 'HTTP Crash Course' — Traversy Media (YouTube)
■ **Coding Practice**	• Use curl or Postman to send GET/POST to a public API (e.g., jsonplaceholder.typicode.com). • Read the raw HTTP request and response headers.
■ **Algorithms**	• LeetCode #704 — Binary Search (quick warm-up). • LeetCode #1 — Two Sum (O(n) solution, 5 minute solve target).
■ **Project Work**	• Setup: create a new ASP.NET Core Web API project (dotnet new webapi -n TaskApi). • Run it. Open the Swagger UI. Understand the auto-generated endpoint.
✍■ **Reflection**	• What is the difference between GET and POST? • Why does HTTP use status codes instead of always returning 200? • What does REST stand for and what are its constraints?

Day 30 | Your First REST API in ASP.NET Core

■ **Concepts**	• Controllers and routing • Action methods • IActionResult • Returning JSON
■ **Theory & Understanding**	• Controller: a class that handles incoming HTTP requests. One controller per resource (e.g., TasksController). • Route: the URL pattern that maps to a method. [HttpGet('/api/tasks')] → GET /api/tasks. • IActionResult: flexible return type — Ok(), NotFound(), BadRequest(), Created(). • ASP.NET Core automatically serializes your C# objects to JSON.
■ **YouTube Videos**	• 'ASP.NET Core REST API' — Amigoscode (YouTube, first 30 mins)
■ **Coding Practice**	• Create TasksController with: GET all tasks, GET task by id, POST new task, DELETE task. • Use an in-memory List as your data store (no database yet). • Test all endpoints in Swagger.

■ **Algorithms**	• LeetCode #53 — Maximum Subarray. • LeetCode #121 — Best Time to Buy and Sell Stock.
■ **Project Work**	• TaskApi: fully working CRUD API in memory. All 4 endpoints tested and returning correct status codes.
✍■ **Reflection**	• What does [ApiController] attribute do? • Why return IActionResult instead of a raw object? • What is Swagger and why is it useful for API development?

Day 31	Models, DTOs & Input Validation
■ Concepts	• Domain models vs DTOs • Data annotations for validation • ModelState • Why DTOs matter
■ Theory & Understanding	• A DTO (Data Transfer Object) is a class used for API input/output — separate from your domain model. • Why? Your domain model might have sensitive fields. DTOs expose only what clients need. • Data annotations: [Required], [MaxLength], [Range] — automatically validate incoming JSON. • ModelState.IsValid: ASP.NET Core checks annotations before your code runs (with [ApiController]).
■ YouTube Videos	• 'DTOs in ASP.NET Core' — IAmTimCorey (YouTube)
■ Coding Practice	• Create: CreateTaskDto { string Title (required, max 100), string Description, DateTime DueDate }. • Add validation: title can't be empty, due date must be in the future. • Test: send invalid input to API and verify 400 Bad Request with error details.
■ Algorithms	• LeetCode #125 — Valid Palindrome. • LeetCode #167 — Two Sum II.
■ Project Work	• TaskApi: add DTOs, validation, proper error responses. No more accepting raw Task objects.
✍■ Reflection	• What is the difference between a model and a DTO? • What happens when validation fails — who handles it? • What is 'over-posting' and how do DTOs prevent it?

Day 32	Databases — SQL Fundamentals
■ Concepts	• Relational databases • Tables, rows, columns • Primary keys, foreign keys • Core SQL: SELECT, INSERT, UPDATE, DELETE, JOIN
■ Theory & Understanding	• A relational database stores data in tables (like spreadsheets) with strict structure. • Primary key: unique identifier for each row. Foreign key: a reference to another table's primary key. • SQL is the language for talking to relational databases. • JOIN combines rows from multiple tables based on related columns. • Think of tables as C# classes, rows as objects, columns as properties.
■ YouTube Videos	• 'SQL Tutorial for Beginners' — Programming with Mosh (YouTube, first 45 mins)
■ Coding Practice	• Install SQLite and DB Browser for SQLite. • Create a Tasks database: Tasks table (Id, Title, Description, IsComplete, DueDate). • Write SQL: SELECT all, SELECT by id, INSERT, UPDATE, DELETE. • Add a Categories table. Join Tasks with Categories.
■ Algorithms	• LeetCode #20 — Valid Parentheses. • LeetCode #155 — Min Stack.

■ **Project Work**	• TaskApi: design the database schema on paper before touching code.
✍■ **Reflection**	• What is the difference between a primary key and a foreign key? • What does JOIN do? Give an analogy. • Why use a database instead of saving to a JSON file?

Day 33 | Entity Framework Core — ORM Basics

■ **Concepts**	• What is an ORM? • DbContext • DbSet • Migrations • CRUD with EF Core
■ **Theory & Understanding**	• ORM (Object-Relational Mapper): translates between C# objects and database rows automatically. • Instead of writing SQL, you work with C# classes. EF Core generates the SQL for you. • DbContext: the gateway to your database. One DbSet per table. • Migration: a versioned change to your database schema. Run dotnet ef migrations add / dotnet ef database update. • Gotcha: EF Core is powerful but can generate inefficient SQL. Always understand what queries it produces.
■ **YouTube Videos**	• 'Entity Framework Core Crash Course' — IAmTimCorey (YouTube)
■ **Coding Practice**	• Add EF Core + SQLite to TaskApi. • Create AppDbContext, configure Task entity. • Run first migration. Verify the database file is created. • Replace in-memory list with EF Core CRUD in TasksController.
■ **Algorithms**	• LeetCode #206 — Reverse Linked List. • LeetCode #21 — Merge Two Sorted Lists.
■ **Project Work**	• TaskApi: fully connected to SQLite via EF Core. Data persists between restarts.
✍■ **Reflection**	• What is the purpose of a migration in EF Core? • How does EF Core map a class to a database table? • What SQL does 'context.Tasks.Where(t => !t.IsComplete).ToList()' generate?

Day 34 | Repository Pattern

■ **Concepts**	• Repository pattern • Why abstract data access • ITaskRepository interface • Testability
■ **Theory & Understanding**	• Repository pattern: put all database access code in dedicated repository classes. • Controllers should NOT contain EF Core queries. Controllers call repositories. • Benefit 1: separation of concerns — controller handles HTTP, repository handles data. • Benefit 2: testability — swap real database with a fake in-memory repository for tests. • Benefit 3: if you change from EF Core to another ORM, only repository changes.
■ **YouTube Videos**	• 'Repository Pattern in ASP.NET Core' — Nick Chapsas (YouTube)
■ **Coding Practice**	• Create ITaskRepository: GetAll, GetById, Create, Update, Delete. • Create EfTaskRepository implementing ITaskRepository. • Register in DI: builder.Services.AddScoped(). • Inject ITaskRepository into TasksController. Remove all direct DbContext calls.

■ **Algorithms**	• LeetCode #141 — Linked List Cycle. • LeetCode #876 — Middle of the Linked List.
■ **Project Work**	• TaskApi: refactored to use repository pattern. Controller is clean HTTP logic only.
✍■ **Reflection**	• What problem does the repository pattern solve? • What is Dependency Injection and why does it matter? • How would you test a controller that uses ITaskRepository?

Day 35	Dependency Injection — Deep Understanding
■ Concepts	• DI container in ASP.NET Core • Service lifetimes: Singleton, Scoped, Transient • Constructor injection • Why DI?
■ Theory & Understanding	• DI: instead of creating objects inside a class (new MyService()), you receive them from outside. • This makes classes loosely coupled — they depend on abstractions, not concretions. • Singleton: one instance for the whole application lifetime. • Scoped: one instance per HTTP request. • Transient: new instance every time it's requested. • 99% of your services will be Scoped. DbContext is always Scoped.
■ YouTube Videos	• 'Dependency Injection in ASP.NET Core' — Nick Chapsas (YouTube)
■ Coding Practice	• Create a logging service (ILogger is built in — use it). • Add a custom INotificationService, register as Scoped, inject into repository. • Try registering a Scoped service as Singleton — observe the error and understand why.
■ Algorithms	• LeetCode #104 — Maximum Depth of Binary Tree. • LeetCode #226 — Invert Binary Tree.
■ Project Work	• TaskApi: add proper logging throughout (ILogger). Log when tasks are created, updated, deleted.
✍■ Reflection	• What happens if you inject a Scoped service into a Singleton? • Why is DI better than manually calling 'new'? • What is the DI container's job in ASP.NET Core?

Day 36	Middleware & Request Pipeline
■ Concepts	• ASP.NET Core pipeline • Middleware • Custom middleware • app.Use vs app.Run
■ Theory & Understanding	• Every HTTP request flows through a pipeline of middleware — each can inspect, modify, or short-circuit. • Order matters: middleware registered first runs first. • Built-in middleware: routing, authentication, CORS, static files, error handling. • app.Use: calls next middleware. app.Run: terminal — doesn't call next. • Analogy: airport security — baggage check, body scan, passport control. Each is middleware.
■ YouTube Videos	• 'ASP.NET Core Middleware' — Nick Chapsas (YouTube)
■ Coding Practice	• Write custom middleware: log request method + path + response status for every request. • Write middleware: add a custom header 'X-App-Version: 1.0' to every response. • Understand and use app.UseExceptionHandler for global error handling.
■ Algorithms	• LeetCode #200 — Number of Islands. • LeetCode #695 — Max Area of Island.
■ Project Work	• TaskApi: add request logging middleware and global exception handling middleware.

✍■ **Reflection**	• What is the difference between app.Use and app.Run? • Why does middleware order matter? • What happens to unhandled exceptions without UseExceptionHandler?

Day 37	Async/Await in C# & Async APIs
■ Concepts	• Async/await • Task • I/O-bound vs CPU-bound • Why async matters for backends
■ Theory & Understanding	• Async: while waiting for slow I/O (database, network), the thread is freed to handle other requests. • Synchronous: thread is BLOCKED while waiting. Async: thread goes back to the pool, returns when data is ready. • async/await keywords make asynchronous code look synchronous — no callback hell. • Rule: if it touches a database, network, or file system → make it async. • All EF Core methods have async versions: ToListAsync(), FirstOrDefaultAsync(), SaveChangesAsync().
■ YouTube Videos	• 'Async/Await in C#' — Nick Chapsas (YouTube)
■ Coding Practice	• Convert all repository methods to async: Task> GetAllAsync(). • Add async to all controller action methods. • Understand: what does 'await' actually do?
■ Algorithms	• LeetCode #70 — Climbing Stairs. • LeetCode #198 — House Robber.
■ Project Work	• TaskApi: fully async — all DB operations use async methods.
✍■ Reflection	• What is the difference between I/O-bound and CPU-bound work? • Why does async improve web server throughput? • What happens if you call an async method without await?

Day 38	Authentication — JWT Basics
■ Concepts	• Authentication vs Authorization • JWT structure • Generating tokens • Validating tokens in ASP.NET Core
■ Theory & Understanding	• Authentication: WHO are you? Authorization: what are you ALLOWED to do? • JWT (JSON Web Token): a compact, signed token. Contains claims (user id, roles) encoded as JSON. • Structure: Header.Payload.Signature — all Base64-encoded and signed with a secret key. • Flow: user logs in → server validates credentials → server returns JWT → client sends JWT in headers → server validates JWT on each request. • Tokens are stateless — server doesn't store sessions. The token itself carries the identity.
■ YouTube Videos	• 'JWT Authentication in ASP.NET Core' — Amigoscode (YouTube)
■ Coding Practice	• Add Microsoft.AspNetCore.Authentication.JwtBearer package. • Create UserController with POST /api/auth/login → returns JWT. • Protect TasksController endpoints with [Authorize]. • Test: access protected endpoint without token (401), then with valid token (200).

■ **Algorithms**	• LeetCode #238 — Product of Array Except Self. • LeetCode #49 — Group Anagrams.
■ **Project Work**	• TaskApi: users must log in to manage tasks. JWT authentication working end-to-end.
✍■ **Reflection**	• What are the 3 parts of a JWT? • What is the difference between authentication and authorization? • Why are JWTs stateless? What are the tradeoffs?

Day 39 | Error Handling & API Best Practices

■ **Concepts**	• Problem Details (RFC 7807) • Global exception handler • Consistent error responses • API versioning basics
■ **Theory & Understanding**	• Good APIs return consistent, machine-readable error responses. • RFC 7807 Problem Details: standard format — type, title, status, detail, instance. • Never expose stack traces or internal errors to clients — it's a security risk. • Log everything internally. Return only relevant info externally. • Versioning: /api/v1/tasks vs /api/v2/tasks — allows breaking changes without breaking clients.
■ **YouTube Videos**	• 'ASP.NET Core Error Handling' — Nick Chapsas (YouTube)
■ **Coding Practice**	• Implement global exception middleware that returns Problem Details format. • Create custom exception classes: NotFoundException, ValidationException. • Test: throw NotFoundException in repository → verify 404 Problem Details response.
■ **Algorithms**	• LeetCode #3 — Longest Substring Without Repeating Characters. • LeetCode #424 — Longest Repeating Character Replacement.
■ **Project Work**	• TaskApi: all errors return consistent Problem Details JSON. No stack traces in production.
✍■ **Reflection**	• What is Problem Details and why is it a standard? • Why should you never expose internal exceptions to API clients? • What HTTP status code should 'item not found' return?

Day 40 | Testing — Unit Tests in C#

■ **Concepts**	• Why testing matters • xUnit basics • Arrange-Act-Assert pattern • Mocking with Moq
■ **Theory & Understanding**	• A unit test verifies one small piece of logic in isolation. • Arrange: set up inputs. Act: call the method. Assert: verify the result. • Mocking: replace real dependencies (like the database) with fake ones that you control. • If code is hard to test, it usually means it's poorly designed (too many responsibilities). • Tests are your safety net — they let you refactor without fear.
■ **YouTube Videos**	• 'Unit Testing in C# with xUnit' — IAmTimCorey (YouTube)
■ **Coding Practice**	• Create TaskApi.Tests project. Add xUnit, Moq. • Write unit tests for: IsPalindrome, FindMax, FizzBuzz logic. • Write tests for: TaskService.CreateTask — mock ITaskRepository. • Aim for at least 10 tests that all pass.
■ **Algorithms**	• LeetCode #347 — Top K Frequent Elements. • LeetCode #75 — Sort Colors.
■ **Project Work**	• TaskApi: unit tests for all service/business logic methods.

✍■ **Reflection**	• What is Arrange-Act-Assert? • Why do we mock dependencies in unit tests? • What is the difference between a unit test and an integration test?

Day 41	API Documentation with Swagger/OpenAPI
■ Concepts	• OpenAPI specification • Swagger UI • XML comments for docs • API contracts
■ Theory & Understanding	• OpenAPI: a standard for describing REST APIs. Swagger UI is a visual interface generated from it. • Good API documentation = faster integration for consumers (frontend, mobile, partners). • XML doc comments on your action methods appear in Swagger automatically. • The API contract is a promise: don't break it without versioning.
■ YouTube Videos	• No specific video — read official ASP.NET Core Swagger docs.
■ Coding Practice	• Add XML documentation comments to all controller methods. • Add response types: [ProducesResponseType(200)], [ProducesResponseType(404)]. • Customize Swagger UI: add title, description, version. • Add JWT support to Swagger so you can test protected endpoints.
■ Algorithms	• LeetCode #704 — Binary Search (3-min solve target). • LeetCode #153 — Find Minimum in Rotated Sorted Array.
■ Project Work	• TaskApi: fully documented Swagger UI. Every endpoint has descriptions and example responses.
✍■ Reflection	• What is an API contract? • Why is documentation as important as the code? • What does [ProducesResponseType] tell Swagger?

Day 42	Phase 3 Review & TaskApi v1 Complete
■ Concepts	• Full Phase 3 review • Code quality audit • Documentation
■ Theory & Understanding	• Today: review, clean up, and complete the TaskApi to production-quality v1. • Quality checklist: async everywhere, proper error handling, validation, auth, tests, Swagger docs.
■ YouTube Videos	• No new video.
■ Coding Practice	• Write from memory: create a new ASP.NET Core endpoint with: model, DTO, validation, async, error handling. • Time yourself — this should feel natural now.
■ Algorithms	• LeetCode #33 — Search in Rotated Sorted Array. • LeetCode #15 — 3Sum (medium — trace on paper first).
■ Project Work	• TaskApi v1 COMPLETE: CRUD, EF Core + SQLite, repository pattern, JWT auth, async, validation, error handling, unit tests, Swagger docs. • Push to GitHub. Write a comprehensive README.

✍■ **Reflection**	• What are the 5 pillars of a production-quality API? • What part of TaskApi am I least confident about? • What would I do differently if starting TaskApi from scratch?

PHASE 4 — Advanced Backend & Full Projects (Days 43–70)

Build real systems. Handle auth, validation, architecture.

Day 43 | Software Architecture Principles

■ **Concepts**	• SOLID principles (full) • Clean Architecture overview • Separation of concerns • Dependency rules
■ **Theory & Understanding**	• SOLID: S=Single Responsibility, O=Open/Closed, L=Liskov Substitution, I=Interface Segregation, D=Dependency Inversion. • Clean Architecture: dependencies point inward. Domain (core) has no dependencies. Infrastructure depends on domain. • Why: code that follows these principles is easier to test, extend, and maintain. • Don't over-engineer from day 1 — understand the principles, apply them when they provide clear value.
■ **YouTube Videos**	• 'SOLID Principles in C#' — Nick Chapsas (YouTube)
■ **Coding Practice**	• Review TaskApi: identify any SOLID violations. • Fix the most obvious one (likely Single Responsibility in a bloated controller).
■ **Algorithms**	• LeetCode #15 — 3Sum. • LeetCode #11 — Container With Most Water.
■ **Project Work**	• Start new project: BookstoreApi. • Design the domain: Book, Author, Category, Order, OrderItem. • Draw the entity relationships before writing any code.
✍■ **Reflection**	• What does Single Responsibility mean for a controller? • Why do dependencies point inward in Clean Architecture? • Explain Open/Closed principle in plain English.

Day 44 | Advanced EF Core — Relationships & Queries

■ **Concepts**	• One-to-many, many-to-many relationships • Navigation properties • Eager loading (Include) • Query optimization
■ **Theory & Understanding**	• EF Core models relationships through navigation properties: Book has a List (many-to-many). • Eager loading: use .Include() to load related entities in a single query. • Lazy loading: loads related data only when accessed (can cause N+1 problem — avoid by default). • N+1 problem: loading 100 books then fetching each book's author separately = 101 queries. Use .Include().
■ **YouTube Videos**	• 'EF Core Relationships' — IAmTimCorey (YouTube)
■ **Coding Practice**	• Configure: Author has many Books. Book has many Categories (many-to-many). • Write queries: all books with their authors, books by category, top-rated books. • View the generated SQL using EF Core logging. Identify any N+1 issues.

■ **Algorithms**	• LeetCode #543 — Diameter of Binary Tree. • LeetCode #572 — Subtree of Another Tree.
■ **Project Work**	• BookstoreApi: EF Core entities with relationships configured. First migration applied.
✍■ **Reflection**	• What is the N+1 problem and how do you prevent it? • When should you use eager vs lazy loading? • What does the generated SQL from EF Core look like for a JOIN query?

Day 45	Service Layer & Business Logic
■ Concepts	• Service layer pattern • Where does business logic go? • CQRS preview • Thin controllers
■ Theory & Understanding	• Service layer: sits between controllers and repositories. Contains business logic. • Rule: controllers should be thin — validate input, call service, return result. • Business logic in services: 'a book can only be ordered if stock > 0', 'users can only have 5 active orders'. • CQRS (Command Query Responsibility Segregation): separate reads from writes. Preview concept for later.
■ YouTube Videos	• 'Service Layer in ASP.NET Core' — IAmTimCorey (YouTube)
■ Coding Practice	• Create IBookService and BookService. • Move business rules from controller into service. • Write unit tests for BookService using mocked IBookRepository.
■ Algorithms	• LeetCode #105 — Construct Binary Tree from Preorder and Inorder. • LeetCode #235 — Lowest Common Ancestor of BST.
■ Project Work	• BookstoreApi: full layered architecture — Controller → Service → Repository → DbContext.
✍■ Reflection	• What is the difference between a service and a repository? • Why should a controller not contain business logic? • Write the sequence: GET /api/books request → ... → database → ... → response.

Day 46	Role-Based Authorization & User Management
■ Concepts	• Claims-based identity • Roles: Admin, User • Policy-based authorization • User registration/login
■ Theory & Understanding	• Claims: key-value pairs in a JWT. Role claim enables role-based access control. • Policy: named rule — 'AdminOnly' = user must have Admin role. • [Authorize(Policy = 'AdminOnly')] — only admins can access this endpoint. • User management: register (hash password), login (verify hash, return JWT), refresh token basics. • NEVER store plain text passwords. Use BCrypt or ASP.NET Core Identity's password hasher.
■ YouTube Videos	• 'Role-Based Authorization ASP.NET Core' — Nick Chapsas (YouTube)
■ Coding Practice	• Add User entity with Role field. • Hash passwords with BCrypt.Net. • Add Admin-only endpoints (e.g., delete any order). • Test: verify non-admin gets 403 Forbidden.
■ Algorithms	• LeetCode #102 — Binary Tree Level Order Traversal (BFS). • LeetCode #199 — Binary Tree Right Side View.

■ **Project Work**	• BookstoreApi: Admin can manage inventory. Users can browse and order. Both protected by JWT + roles.
✍■ **Reflection**	• What is the difference between a claim and a role? • What HTTP status code means 'authenticated but not allowed'? • Why must passwords be hashed and not encrypted?

Day 47	Pagination, Filtering & Sorting APIs
■ Concepts	• Query parameters • Pagination (page/pageSize) • Filtering by field • Sorting by column • HATEOAS preview
■ Theory & Understanding	• APIs that return all data at once are unusable at scale. Always paginate. • Query params: GET /api/books?page=2&pageSize;=10&category;=fiction&sortBy;=price■=asc. • Return metadata: total count, page, pageSize, total pages. • Use SKIP/TAKE in EF Core: .Skip((page-1)*pageSize).Take(pageSize).
■ YouTube Videos	• 'Pagination in ASP.NET Core' — Nick Chapsas (YouTube)
■ Coding Practice	• Create a generic PagedResult class. • Implement: GET /api/books with pagination, filtering by author/category, sorting by price/title. • Test with Postman: various combinations of query params.
■ Algorithms	• LeetCode #300 — Longest Increasing Subsequence. • LeetCode #322 — Coin Change.
■ Project Work	• BookstoreApi: all list endpoints paginated with filtering and sorting.
✍■ Reflection	• Why is pagination important for large datasets? • What is SKIP/TAKE and how does it map to SQL OFFSET/LIMIT? • What metadata should a paginated API response include?

Day 48	Caching — In-Memory & Distributed
■ Concepts	• Why cache? • IMemoryCache • IDistributedCache • Cache invalidation • Redis preview
■ Theory & Understanding	• Cache: store expensive computation results in fast memory so repeated requests are instant. • In-memory cache: fast, but lost on restart, not shared across multiple servers. • Distributed cache (Redis): shared across all server instances, survives restarts. • Cache invalidation: the hard part — when to clear the cache so stale data doesn't serve. • Rule: cache read-heavy, infrequently-changing data. Never cache user-specific sensitive data in shared cache.
■ YouTube Videos	• 'Caching in ASP.NET Core' — Nick Chapsas (YouTube)
■ Coding Practice	• Add IMemoryCache to BookstoreApi. • Cache the full books list with a 5-minute expiry. • Invalidate cache when a book is updated or deleted.
■ Algorithms	• LeetCode #139 — Word Break. • LeetCode #152 — Maximum Product Subarray.
■ Project Work	• BookstoreApi: cached book listings. Measure response time before and after caching.

✍■ **Reflection**	• What is cache invalidation and why is it called 'the hardest problem'? • When would in-memory cache fail you? When do you need Redis? • What data should NEVER be cached?

Day 49	Background Jobs & Hosted Services
■ Concepts	• IHostedService • BackgroundService • Scheduled tasks • Hangfire preview
■ Theory & Understanding	• Some work shouldn't happen synchronously during an HTTP request: sending emails, generating reports, cleaning up data. • BackgroundService: runs in the background for the lifetime of the application. • Use for: periodic cleanup, sending notifications, syncing data. • Hangfire: library for persistent, scheduled background jobs with a dashboard.
■ YouTube Videos	• 'Background Services in ASP.NET Core' — Nick Chapsas (YouTube)
■ Coding Practice	• Create a BackgroundService that logs 'System alive' every 30 seconds. • Create a service that clears expired orders from the DB every hour.
■ Algorithms	• LeetCode #416 — Partition Equal Subset Sum. • LeetCode #62 — Unique Paths.
■ Project Work	• BookstoreApi: add background job to send daily low-stock alerts (to console for now).
✍■ Reflection	• When should work be done in a background service vs inline in a request? • What is the difference between IHostedService and BackgroundService? • What are the risks of long-running tasks inside HTTP requests?

Day 50	Midpoint Review — Milestone Day
■ Concepts	• Review all concepts to date • Project audit • Skill assessment
■ Theory & Understanding	• You are halfway through the 90 days. This is a critical checkpoint. • Today: honestly assess what you know well, what is shaky, and what you have not practised.
■ YouTube Videos	• Re-watch one video from the past 2 weeks that still feels unclear.
■ Coding Practice	• Write from scratch: a complete CRUD API (no looking) — controller, service, repository, DTO, validation. • Time yourself. Target: 90 minutes.
■ Algorithms	• LeetCode #5 — Longest Palindromic Substring. • LeetCode #647 — Palindromic Substrings.
■ Project Work	• Audit TaskApi and BookstoreApi against quality checklist. Fix any gaps.
✍■ Reflection	• What are the 5 things I am most confident about? • What are the 3 things I am weakest on? • What is my adjusted plan for the remaining 40 days?

Day 51	SignalR — Real-Time Communication
■ Concepts	• WebSockets basics • SignalR Hubs • Groups and connections • Real-time use cases
■ Theory & Understanding	• HTTP is request-response — the client must ask to receive data. WebSockets maintain an open connection. • SignalR abstracts WebSockets. Server can push data to clients without them asking. • Use cases: live chat, notifications, live dashboards, collaborative tools. • Hub: the server-side class clients connect to. Clients call methods on it. Server calls methods on clients.
■ YouTube Videos	• 'SignalR in ASP.NET Core' — Amigoscode (YouTube)
■ Coding Practice	• Add SignalR to BookstoreApi. • Create NotificationHub. • When a new order is placed, push notification to all connected admin clients.
■ Algorithms	• LeetCode #91 — Decode Ways. • LeetCode #213 — House Robber II.
■ Project Work	• BookstoreApi: real-time order notifications to admins via SignalR.
✍■ Reflection	• What is the difference between HTTP polling and WebSockets? • When should you use SignalR vs a message queue? • What is a SignalR Hub?

Day 52	Advanced LINQ & Expression Trees
■ Concepts	• Complex LINQ queries • Dynamic filtering • Expression> • Building query specs
■ Theory & Understanding	• Expression trees: represent code as data. EF Core uses them to translate LINQ to SQL. • Dynamic filtering: build queries at runtime based on user input without string concatenation. • Specification pattern: encapsulate query criteria in a reusable class. • Never concatenate SQL strings — this is SQL injection waiting to happen.
■ YouTube Videos	• 'Advanced LINQ in C#' — Nick Chapsas (YouTube)
■ Coding Practice	• Build a generic FilteredQuery that accepts a list of criteria and builds an EF Core query. • Test: filter books by price range, author, in-stock, published after year X — all dynamically.
■ Algorithms	• LeetCode #337 — House Robber III (tree DP). • LeetCode #279 — Perfect Squares.
■ Project Work	• BookstoreApi: advanced search endpoint supporting multiple optional filter parameters.
✍■ Reflection	• What is an Expression tree vs a Func? • What is SQL injection and how does EF Core prevent it? • What is the Specification pattern and when is it useful?

Day 53	Integration Testing
■ Concepts	• Integration vs unit tests • WebApplicationFactory • Testing HTTP endpoints • Test database setup
■ Theory & Understanding	• Integration tests test multiple layers together — controller + service + repository + database. • WebApplicationFactory: spins up a test version of your ASP.NET Core app in memory. • Use a separate in-memory SQLite database for tests — never test against production data. • HttpClient: makes real HTTP requests to your test server. Assert on status codes and response body.
■ YouTube Videos	• 'Integration Testing ASP.NET Core' — Nick Chapsas (YouTube)
■ Coding Practice	• Create BookstoreApi.IntegrationTests project. • Write tests: POST /api/books creates a book (assert 201 + body), GET /api/books returns list. • Write: protected endpoint returns 401 without token, 200 with valid token.
■ Algorithms	• LeetCode #73 — Set Matrix Zeroes. • LeetCode #54 — Spiral Matrix.
■ Project Work	• BookstoreApi: integration test suite covering all major endpoints.
✍■ Reflection	• What is the difference between unit and integration tests? • Why use a separate database for tests? • What should an integration test NOT do?

Day 54	API Security — CORS, Rate Limiting, HTTPS
■ Concepts	• CORS policy • Rate limiting • HTTPS enforcement • Security headers
■ Theory & Understanding	• CORS (Cross-Origin Resource Sharing): controls which frontend origins can call your API. • Rate limiting: prevent abuse — limit to e.g. 100 requests/minute per IP. • HTTPS: encrypt all traffic in transit. Never accept sensitive data over HTTP. • Security headers: X-Content-Type-Options, X-Frame-Options, Content-Security-Policy.
■ YouTube Videos	• 'API Security Best Practices' — Fireship (YouTube)
■ Coding Practice	• Configure CORS to allow only specific origins. • Add rate limiting middleware: 100 req/min per IP. • Add security headers middleware. • Force HTTPS redirect.
■ Algorithms	• LeetCode #48 — Rotate Image. • LeetCode #128 — Longest Consecutive Sequence.
■ Project Work	• BookstoreApi: hardened security — CORS, rate limiting, HTTPS, security headers.

✍■ **Reflection**	• What is a CORS error and why does it only happen in browsers? • What is a brute-force attack and how does rate limiting help? • What is HTTPS and why is HTTP not sufficient for APIs?

Day 55	Database Design & SQL Advanced
■ Concepts	• Normalization (1NF, 2NF, 3NF) • Indexing • Transactions • Stored procedures vs ORM
■ Theory & Understanding	• Normalization: eliminate data redundancy. 1NF: atomic values. 2NF: no partial dependency. 3NF: no transitive dependency. • Index: speeds up queries on a column — like a book index. Cost: slows inserts/updates. • Add indexes on frequently-queried columns: foreign keys, search fields. • Transaction: group operations that must ALL succeed or ALL fail. Ensures consistency.
■ YouTube Videos	• 'Database Indexing Explained' — Hussein Nasser (YouTube)
■ Coding Practice	• Add indexes to BookstoreApi: on Book.CategoryId, Book.AuthorId, Book.Title. • Wrap order creation in a transaction: decrement stock + create order = atomic. • View query plans using EF Core query logging.
■ Algorithms	• LeetCode #207 — Course Schedule (graph, topological sort). • LeetCode #210 — Course Schedule II.
■ Project Work	• BookstoreApi: optimised database — indexes, transactions for orders.
✍■ Reflection	• What is a database index and what is its trade-off? • What is a database transaction and why do you need it for orders? • What does 3NF mean in plain English?

Day 56	Docker & Containerization Basics
■ Concepts	• What is Docker? • Dockerfile • docker-compose • Running ASP.NET Core in Docker
■ Theory & Understanding	• Docker: package your app with all its dependencies into a container — runs the same everywhere. • Dockerfile: instructions for building a container image. • docker-compose: define multi-container apps (app + database) in one file. • Every modern backend job expects Docker knowledge. This is non-negotiable.
■ YouTube Videos	• 'Docker for Beginners' — TechWorld with Nana (YouTube, first 30 mins)
■ Coding Practice	• Write a Dockerfile for TaskApi. • Build and run: docker build -t taskapi . && docker run -p 8080:80 taskapi. • Write docker-compose.yml: taskapi + postgres database.
■ Algorithms	• LeetCode #994 — Rotting Oranges (BFS). • LeetCode #417 — Pacific Atlantic Water Flow.
■ Project Work	• TaskApi: runs in Docker. docker-compose up starts both API and database.

✍■ **Reflection**	• What problem does Docker solve? • What is the difference between an image and a container? • What is docker-compose and when do you use it?

Day 57 | CI/CD Pipeline Basics

■ **Concepts**	• What is CI/CD? • GitHub Actions • Build, test, deploy pipeline • Environment variables
■ **Theory & Understanding**	• CI (Continuous Integration): automatically build and test every commit. • CD (Continuous Deployment): automatically deploy to production when tests pass. • GitHub Actions: workflow YAML files in .github/workflows/ — free for public repos. • Every push → build → run tests → if all pass → deploy. This is the professional standard.
■ **YouTube Videos**	• 'GitHub Actions CI/CD for .NET' — Nick Chapsas (YouTube)
■ **Coding Practice**	• Create .github/workflows/ci.yml for TaskApi. • Steps: checkout → setup dotnet → restore → build → test. • Push a breaking change — watch CI catch it.
■ **Algorithms**	• LeetCode #133 — Clone Graph. • LeetCode #261 — Graph Valid Tree.
■ **Project Work**	• TaskApi: CI pipeline runs on every push. Build fails if tests fail.
✍■ **Reflection**	• What is the difference between CI and CD? • What happens in your CI pipeline on a broken commit? • Why are environment variables important in CI/CD?

Day 58 | Logging, Monitoring & Observability

■ **Concepts**	• Structured logging with Serilog • Log levels • Health checks • Application metrics basics
■ **Theory & Understanding**	• Structured logging: log as key-value pairs (JSON), not plain strings. Makes searching logs possible. • Log levels: Trace, Debug, Information, Warning, Error, Critical. Set minimum level per environment. • Health checks: /health endpoint that tells orchestrators if the app is running correctly. • Observability: logs + metrics + traces. You can't fix what you can't see.
■ **YouTube Videos**	• 'Serilog in ASP.NET Core' — Nick Chapsas (YouTube)
■ **Coding Practice**	• Replace default logging with Serilog. Output to console AND file. • Log structured data: include UserId, RequestId in every log. • Add health check endpoint: checks DB connection.
■ **Algorithms**	• LeetCode #126 — Word Ladder II (BFS — hard, understand approach). • LeetCode #127 — Word Ladder (BFS).
■ **Project Work**	• BookstoreApi: Serilog structured logging, health check endpoint.

✍■ **Reflection**	• What is the difference between structured and unstructured logging? • What log level would you use for 'user not found'? For 'database connection failed'? • What is a health check and why does Kubernetes need it?

Day 59	Advanced Algorithm Practice Day
■ Concepts	• Mixed algorithm patterns • Problem-solving strategy • Time management under pressure
■ Theory & Understanding	• Today is pure algorithmic practice. This is interview simulation. • Strategy: read problem → state the pattern (sliding window? DP? graph?) → pseudocode → code → verify edge cases. • Time limit: 25 minutes per problem. If stuck at 25 mins, look at the approach (not solution), then finish.
■ YouTube Videos	• No video — pure practice.
■ Coding Practice	• No project work today. Full focus on algorithms.
■ Algorithms	• LeetCode #76 — Minimum Window Substring (hard — sliding window). • LeetCode #42 — Trapping Rain Water (two pointers or stack). • LeetCode #23 — Merge K Sorted Lists. • LeetCode #297 — Serialize and Deserialize Binary Tree.
■ Project Work	• No project work today.
✍■ Reflection	• Which problem took the longest and why? • What pattern did I recognize immediately? What took time? • Am I improving at reading problems quickly?

Day 60	BookstoreApi — Feature Completion
■ Concepts	• API completeness • End-to-end testing • Documentation
■ Theory & Understanding	• Completeness check: does the API do what it promises? Are all edge cases handled? Is it documented? • End-to-end: use Postman to simulate a full user journey — register → login → browse → order → admin fulfills.
■ YouTube Videos	• No new video.
■ Coding Practice	• Write a Postman collection documenting the complete user journey. • Run through all scenarios manually and fix any bugs.
■ Algorithms	• LeetCode #56 — Merge Intervals. • LeetCode #57 — Insert Interval.
■ Project Work	• BookstoreApi COMPLETE: all features working, tested, documented, containerized. • Push to GitHub. Write comprehensive README.
✍■ Reflection	• What is the most complex feature in BookstoreApi? • What would I add if I had two more weeks? • What technical debt exists and what is the plan to address it?

Day 61	Microservices — Introduction & Concepts
■ Concepts	• Monolith vs microservices • Service boundaries • Inter-service communication • When NOT to use microservices
■ Theory & Understanding	• Monolith: one deployable unit containing all features. Simple to start, hard to scale. • Microservices: multiple small, independently deployable services, each owning its data. • Communication: synchronous (HTTP/gRPC) or asynchronous (message queues — RabbitMQ, Kafka). • Warning: microservices add enormous complexity. Don't start with them. Understand monolith first. • When to consider: team size > 10, different scaling needs per feature, independent deployment required.
■ YouTube Videos	• 'Microservices Explained' — TechWorld with Nana (YouTube)
■ Coding Practice	• Sketch on paper: how would BookstoreApi split into microservices? • Identify the service boundaries: Catalog, Orders, Users, Notifications. • No code today — design thinking only.
■ Algorithms	• LeetCode #78 — Subsets. • LeetCode #90 — Subsets II.
■ Project Work	• No new project today — conceptual design day.
✍■ Reflection	• What are the main advantages of microservices? • What are the main disadvantages? • When is a monolith the better choice?

Day 62	Message Queues & Event-Driven Architecture
■ Concepts	• Message queues • RabbitMQ basics • Producer/Consumer pattern • Event-driven design
■ Theory & Understanding	• Message queue: decouple services. Producer sends a message. Consumer processes it asynchronously. • Benefits: services don't need to know about each other, failures are isolated. • RabbitMQ: popular open-source message broker. Exchanges route messages to queues. • Event: something that happened (OrderPlaced). Other services react to events without the publisher knowing.
■ YouTube Videos	• 'RabbitMQ Tutorial' — TechWorld with Nana (YouTube)
■ Coding Practice	• Add RabbitMQ to docker-compose. • Publisher: when order is placed in BookstoreApi, publish OrderPlaced event. • Consumer: simple console app that listens and logs 'Order received: {id}'.
■ Algorithms	• LeetCode #46 — Permutations. • LeetCode #79 — Word Search (backtracking).

■ **Project Work**	• BookstoreApi: event publishing to RabbitMQ on order creation.
✍■ **Reflection**	• What is the difference between synchronous and asynchronous service communication? • What is a dead letter queue? • When would you choose message queues over direct HTTP calls between services?

Day 63 | GraphQL — Alternative to REST

■ **Concepts**	• What is GraphQL? • Schema and resolvers • Queries and mutations • REST vs GraphQL
■ **Theory & Understanding**	• GraphQL: query language for APIs — clients specify exactly what data they need. • REST: server determines the shape of the response. GraphQL: client determines it. • Advantages: no over-fetching, no under-fetching, strongly typed schema. • Disadvantages: more complex caching, learning curve, not always better than REST. • Use case: APIs consumed by multiple clients with different data needs (web, mobile).
■ **YouTube Videos**	• 'GraphQL Explained' — Fireship (YouTube, 10 mins)
■ **Coding Practice**	• Add Hot Chocolate (GraphQL library) to a new minimal project. • Define a Book type and query: { books { title author { name } } }. • Add a mutation: createBook.
■ **Algorithms**	• LeetCode #39 — Combination Sum. • LeetCode #40 — Combination Sum II.
■ **Project Work**	• No BookstoreApi change today — standalone GraphQL exploration project.
✍■ **Reflection**	• What problem does GraphQL solve that REST doesn't? • When would you choose REST over GraphQL? • What is a GraphQL resolver?

Day 64 | Redis — Advanced Caching & Session

■ **Concepts**	• Redis data structures • Caching strategies: cache-aside, write-through • Session storage • Redis as message broker
■ **Theory & Understanding**	• Redis: in-memory data store — extremely fast. Supports strings, lists, sets, sorted sets, hashes. • Cache-aside: app checks cache → if miss, fetches from DB, stores in cache, returns. • Write-through: write to cache AND database simultaneously. • TTL (Time To Live): auto-expire cache entries after N seconds. • Redis can also act as a simple message queue (Pub/Sub) — lightweight alternative to RabbitMQ.
■ **YouTube Videos**	• 'Redis Crash Course' — Fireship (YouTube)
■ **Coding Practice**	• Add Redis to docker-compose. • Replace IMemoryCache with IDistributedCache (Redis backend). • Implement cache-aside pattern for book listings. • Set appropriate TTLs for different data types.
■ **Algorithms**	• LeetCode #51 — N-Queens (backtracking). • LeetCode #37 — Sudoku Solver.
■ **Project Work**	• BookstoreApi: Redis caching with cache-aside pattern.

✍■ **Reflection**	• What is the difference between cache-aside and write-through? • What TTL would you set for book listings? User sessions? Why? • How does Redis handle data persistence?

Day 65	gRPC — High-Performance APIs
■ Concepts	• What is gRPC? • Protocol Buffers • Service definition • gRPC vs REST
■ Theory & Understanding	• gRPC: Google's high-performance RPC framework. Uses HTTP/2 and Protocol Buffers (binary format). • Faster than REST for service-to-service communication: binary protocol, multiplexing. • Define service in a .proto file: service, rpc methods, message types. • Best use: internal microservice communication where performance matters. • Not for public APIs or browser clients without a proxy (gRPC-Web).
■ YouTube Videos	• 'gRPC in ASP.NET Core' — Nick Chapsas (YouTube)
■ Coding Practice	• Create a gRPC service: BookCatalog.proto with GetBook and ListBooks methods. • Implement the gRPC server in ASP.NET Core. • Create a gRPC client (console app) that calls the service.
■ Algorithms	• LeetCode #84 — Largest Rectangle in Histogram. • LeetCode #85 — Maximal Rectangle.
■ Project Work	• Standalone gRPC exploration — catalog service that BookstoreApi could call internally.
✍■ Reflection	• What makes gRPC faster than REST? • What is a .proto file? • When would you use gRPC instead of REST?

Day 66	Clean Architecture — Full Implementation
■ Concepts	• 4 layers: Domain, Application, Infrastructure, Presentation • CQRS with MediatR • Domain events • Dependency flow
■ Theory & Understanding	• Clean Architecture explicitly separates layers with strict dependency rules. • Domain: business entities, rules, no dependencies. • Application: use cases (commands/queries) — depends only on Domain. • Infrastructure: DB, email, external APIs — depends on Application interfaces. • Presentation (API): depends on Application. Never on Infrastructure directly. • MediatR: implements CQRS in-process. Handler per command/query.
■ YouTube Videos	• 'Clean Architecture with ASP.NET Core' — Amichai Mantinband (YouTube)
■ Coding Practice	• Restructure a feature (e.g., 'Create Order') into Clean Architecture. • CreateOrderCommand → CreateOrderCommandHandler → IOrderRepository (interface in Application, implementation in Infrastructure).
■ Algorithms	• LeetCode #295 — Find Median from Data Stream. • LeetCode #146 — LRU Cache.
■ Project Work	• Start refactoring BookstoreApi toward Clean Architecture structure — at least 2 features fully restructured.

✍■ **Reflection**	• Which layer should NOT know about HTTP? • What is CQRS and what problem does it solve? • What is MediatR and how does it work?

Day 67	Performance Optimization
■ Concepts	• Profiling with dotnet-trace • Async optimization • EF Core query optimization • Connection pooling
■ Theory & Understanding	• Rule 1: never optimize without measuring. Profile first, then fix. • Common ASP.NET Core bottlenecks: sync calls in async context, N+1 queries, no pagination, large payloads. • dotnet-trace and dotnet-counters: built-in profiling tools. • Connection pooling: reuse database connections instead of creating new ones per request. • Response compression: gzip/brotli reduces payload size significantly.
■ YouTube Videos	• 'ASP.NET Core Performance' — Nick Chapsas (YouTube)
■ Coding Practice	• Add response compression middleware. • Add output caching (new in .NET 7+) to heavy GET endpoints. • Use AsNoTracking() in EF Core for read-only queries (significant performance gain).
■ Algorithms	• LeetCode #253 — Meeting Rooms II. • LeetCode #621 — Task Scheduler.
■ Project Work	• BookstoreApi: performance tuned — compression, AsNoTracking, output caching on catalog endpoints.
✍■ Reflection	• What does AsNoTracking() do and when should you use it? • What is response compression and how does it help? • What tool would you use to find a performance bottleneck in ASP.NET Core?

Day 68	System Design Fundamentals
■ Concepts	• Scalability: vertical vs horizontal • Load balancing • Database replication • CAP theorem
■ Theory & Understanding	• Vertical scaling: bigger machine. Simple but has limits. • Horizontal scaling: more machines. Requires stateless apps (no in-memory sessions). • Load balancer: distributes requests across multiple server instances. • Database replication: primary (writes) + replicas (reads). Improves read throughput. • CAP theorem: a distributed system can only guarantee 2 of: Consistency, Availability, Partition tolerance.
■ YouTube Videos	• 'System Design Basics' — Gaurav Sen (YouTube)
■ Coding Practice	• No code today. This is a design thinking day. • Design on paper: how would you scale BookstoreApi to 1 million users? • Consider: load balancer, multiple API instances, read replicas, Redis cache, CDN.
■ Algorithms	• LeetCode #994 — Rotting Oranges (BFS practice). • LeetCode #329 — Longest Increasing Path in Matrix.
■ Project Work	• Write a 1-page architecture document for 'BookstoreApi at scale'. Include a diagram.

✍■ **Reflection**	• What is the CAP theorem in plain English? • Why must an API be stateless to scale horizontally? • What is the job of a load balancer?

Day 69 | Advanced Algorithm Practice II

■ **Concepts**	• Hard problems • Problem-solving under pressure • Pattern recognition speed
■ **Theory & Understanding**	• Today: interview simulation. 25 minutes per problem. Think out loud (write thoughts in comments). • After each problem: did I recognize the pattern? Was my complexity analysis correct?
■ **YouTube Videos**	• No video.
■ **Coding Practice**	• No project work. Pure algorithms.
■ **Algorithms**	• LeetCode #124 — Binary Tree Maximum Path Sum. • LeetCode #212 — Word Search II (Trie + backtracking). • LeetCode #239 — Sliding Window Maximum. • LeetCode #32 — Longest Valid Parentheses.
■ **Project Work**	• No project work today.
✍■ **Reflection**	• Which problem did I solve confidently? • Which problem did I struggle with? What was the gap? • What patterns do I need more practice on?

Day 70 | Phase 4 Review & BookstoreApi Final

■ **Concepts**	• Full Phase 4 consolidation • Portfolio preparation • Technical writing
■ **Theory & Understanding**	• Today: complete, polish, and document BookstoreApi to a level you are proud to show an employer. • Your GitHub portfolio is your proof of work. Make it impressive.
■ **YouTube Videos**	• No new video.
■ **Coding Practice**	• Final code review of both projects: clean code, no dead code, consistent naming. • Ensure all tests pass. CI pipeline is green.
■ **Algorithms**	• LeetCode #4 — Median of Two Sorted Arrays (hard). • LeetCode #25 — Reverse Nodes in K-Group (hard).
■ **Project Work**	• BookstoreApi FINAL: comprehensive README (architecture, setup, API docs, design decisions). • Both TaskApi and BookstoreApi on GitHub with green CI badges.
✍■ **Reflection**	• Would I be proud to show this code to a senior engineer? • What is the most technically impressive thing I built in Phase 4? • Am I ready for Phase 5 — interview prep?

PHASE 5 — Polish, Interview Prep & Portfolio (Days 71–90)

Sharpen your edge. Present your best work.

Day 71 | Interview Preparation Strategy

■ **Concepts**	• Types of interviews • Technical screen • System design interview • Behavioural interview
■ **Theory & Understanding**	• Technical screen: 45-60 min coding problem. Medium LeetCode level. Think out loud. • System design: 45-60 min designing a large-scale system. No single right answer. • Behavioural: STAR format (Situation, Task, Action, Result). Prepare 5-7 stories. • Portfolio review: walk through your projects. Know every line of your own code.
■ **YouTube Videos**	• 'How to Prepare for Technical Interviews' — CS Dojo (YouTube)
■ **Coding Practice**	• Solve 3 medium LeetCode problems in 60 minutes total (20 min each). • Record yourself explaining your approach (voice memo is fine).
■ **Algorithms**	• LeetCode #102 — Binary Tree Level Order Traversal. • LeetCode #236 — Lowest Common Ancestor of Binary Tree. • LeetCode #98 — Validate Binary Search Tree.
■ **Project Work**	• Prepare a 5-minute project walkthrough for TaskApi and BookstoreApi.
✍■ **Reflection**	• What is the STAR framework? • Prepare answers to: 'Tell me about a challenge you faced'. • What 3 technical questions am I most afraid to be asked?

Day 72 | LeetCode Blitz — Array & String Patterns

■ **Concepts**	• Consolidate all array/string patterns • Speed and accuracy
■ **Theory & Understanding**	• Today is high-intensity practice. All arrays and string problems from common interview lists. • For each: state pattern → pseudocode → code → complexity. Time: 20 min max per problem.
■ **YouTube Videos**	• No video.
■ **Coding Practice**	• No project work.
■ **Algorithms**	• LeetCode #238 — Product of Array Except Self. • LeetCode #41 — First Missing Positive. • LeetCode #128 — Longest Consecutive Sequence. • LeetCode #76 — Minimum Window Substring. • LeetCode #3 — Longest Substring Without Repeating Characters (5-min target).
■ **Project Work**	• No project work.
✍■ **Reflection**	• Which pattern is now automatic for me? • Which pattern still needs more practice? • Am I writing clean, readable code under pressure?

Day 73	LeetCode Blitz — Trees & Graphs
■ Concepts	• Tree and graph patterns at speed
■ Theory & Understanding	• Strategy: DFS for path problems, BFS for shortest path/levels, backtracking for exploration.
■ YouTube Videos	• No video.
■ Coding Practice	• No project work.
■ Algorithms	• LeetCode #210 — Course Schedule II (topological sort). • LeetCode #417 — Pacific Atlantic Water Flow. • LeetCode #124 — Binary Tree Maximum Path Sum. • LeetCode #297 — Serialize and Deserialize Binary Tree. • LeetCode #572 — Subtree of Another Tree.
■ Project Work	• No project work.
✍■ Reflection	• DFS vs BFS: when do you automatically reach for each? • What is topological sort and what problems need it? • Did I write from memory or need hints?

Day 74	LeetCode Blitz — Dynamic Programming
■ Concepts	• DP patterns at speed • Recognize subproblem structure
■ Theory & Understanding	• DP recognition: 'find the maximum/minimum/count of ways' + 'choices affect future' = likely DP. • Always try tabulation (bottom-up) — cleaner, no recursion stack risk.
■ YouTube Videos	• No video.
■ Coding Practice	• No project work.
■ Algorithms	• LeetCode #322 — Coin Change. • LeetCode #300 — Longest Increasing Subsequence. • LeetCode #416 — Partition Equal Subset Sum. • LeetCode #72 — Edit Distance. • LeetCode #10 — Regular Expression Matching (hard).
■ Project Work	• No project work.
✍■ Reflection	• What is the DP pattern for Coin Change? • How does Edit Distance work — explain it without code. • Which DP problem is still not intuitive?

Day 75	System Design Practice — URL Shortener
■ Concepts	• System design framework • URL shortener design • Requirements, scale, components
■ Theory & Understanding	• Framework: Functional requirements → Non-functional → Estimation → API design → Data model → High-level design → Deep dives. • URL shortener: given a long URL, return a short one. Redirect short → long. • Key decisions: hash function, collision handling, database choice, cache strategy, expiry. • Scale: 100M URLs, 1000 req/sec reads, 100 req/sec writes.
■ YouTube Videos	• 'Design URL Shortener' — Gaurav Sen (YouTube)
■ Coding Practice	• Build a working mini URL shortener in ASP.NET Core — POST /shorten, GET /{code} → redirect.
■ Algorithms	• LeetCode #208 — Implement Trie. • LeetCode #211 — Design Add and Search Words Data Structure.
■ Project Work	• URL Shortener mini-project: working, clean, documented.
✍■ Reflection	• What hash function would you use and why? • How would you handle hash collisions? • How would you scale this to 1 billion URLs?

Day 76	System Design Practice — Twitter Feed / News Feed
■ Concepts	• Feed generation • Fan-out on write vs read • Denormalization for performance
■ Theory & Understanding	• News feed: show the user a personalized stream of posts from people they follow. • Fan-out on write: when user posts, push to all followers' feeds immediately. Fast reads, expensive writes. • Fan-out on read: compute feed at read time from followings. Cheap writes, expensive reads. • Hybrid: fan-out on write for regular users, fan-out on read for celebrity users (millions of followers).
■ YouTube Videos	• 'Design Twitter' — Gaurav Sen (YouTube)
■ Coding Practice	• Design the data model on paper. Sketch the API.
■ Algorithms	• LeetCode #355 — Design Twitter. • LeetCode #1146 — Snapshot Array.
■ Project Work	• No new project — design day.
✍■ Reflection	• What is fan-out on write and what is its trade-off? • What is denormalization and when does it help performance? • How does Twitter handle celebrity accounts differently?

Day 77 | Behavioural Interview Preparation

■ **Concepts**	• STAR framework • Common interview questions • Demonstrating growth
■ **Theory & Understanding**	• Situation: context. Task: what you were responsible for. Action: exactly what YOU did. Result: measurable outcome. • Prepare stories for: challenge overcome, conflict resolved, mistake made (and learned from), leadership, going above and beyond. • Employers aren't just testing skills — they're testing communication, ownership, and growth mindset.
■ **YouTube Videos**	• 'Behavioural Interview Questions' — CS Dojo (YouTube)
■ **Coding Practice**	• Solve 2 easy LeetCode problems in 10 minutes each (warm up habit).
■ **Algorithms**	• LeetCode #217 — Contains Duplicate (5-min target). • LeetCode #242 — Valid Anagram (5-min target).
■ **Project Work**	• Write out 5 full STAR stories in writing. Read them out loud. Time yourself.
✍■ **Reflection**	• Can I explain my projects clearly without technical jargon to a non-technical interviewer? • What is my proudest technical achievement from the past 77 days? • What mistake did I make and what did I learn from it?

Day 78 | Portfolio — GitHub & Resume

■ **Concepts**	• GitHub profile optimization • README standards • Resume writing for backend engineers
■ **Theory & Understanding**	• Your GitHub IS your portfolio. Employers WILL look at your code. • Pinned repos: only your best 3-4 projects. Each must have a README with: what it is, tech stack, how to run, screenshots/demo. • Resume: one page, results-focused bullets, technologies listed, links to GitHub and LinkedIn.
■ **YouTube Videos**	• 'Developer Portfolio Tips' — Traversy Media (YouTube)
■ **Coding Practice**	• Write or improve READMEs for TaskApi and BookstoreApi. • Add architecture diagram using draw.io or Mermaid.
■ **Algorithms**	• LeetCode #79 — Word Search. • LeetCode #212 — Word Search II.
■ **Project Work**	• GitHub profile: bio, pinned repos, all READMEs polished.
✍■ **Reflection**	• Would I click on my own GitHub if I were a recruiter? • Does my resume demonstrate growth and real skills? • What is missing from my portfolio that I should add?

Day 79	Mock Technical Interview — Round 1
■ Concepts	• Simulated interview conditions • Think out loud • Time management
■ Theory & Understanding	• Rules: timer set for 45 minutes. One medium LeetCode problem. No hints. • You MUST think out loud throughout — explain what you're considering, why you reject approaches. • After: honest self-evaluation. What would an interviewer think?
■ YouTube Videos	• No video — full interview simulation.
■ Coding Practice	• Pick a RANDOM medium LeetCode problem. Set 45-minute timer. Solve it. • Write out your thought process in comments as you go.
■ Algorithms	• One randomly selected medium problem — no cherry-picking.
■ Project Work	• No project work.
✍■ Reflection	• Did I communicate my approach before coding? • Did I handle edge cases? • What would I do differently if this were a real interview?

Day 80	Mock Technical Interview — Round 2
■ Concepts	• Same format as Day 79 • Improvement focus
■ Theory & Understanding	• Apply all lessons from Day 79. This time: start by clarifying requirements, state complexity after each approach.
■ YouTube Videos	• No video.
■ Coding Practice	• Another random medium problem. 45-minute timer. • Before coding: write pseudocode. After coding: test with edge cases.
■ Algorithms	• One randomly selected medium problem.
■ Project Work	• No project work.
✍■ Reflection	• Compared to Day 79: did I improve? • Was I calmer? Did I communicate better? • What one habit will I practice before every interview?

Day 81	System Design Mock Interview — Social Media Platform
■ Concepts	• Full system design walkthrough • Scaling decisions • Trade-off discussion
■ Theory & Understanding	• Design: Instagram-like photo sharing platform. • Cover: functional requirements, API design, database schema (relational? NoSQL?), storage (S3), CDN for media, feed generation, search. • Practice: set a 45-minute timer. Go through the framework without looking at notes.
■ YouTube Videos	• 'Design Instagram' — Gaurav Sen (YouTube) — watch AFTER your attempt.
■ Coding Practice	• Draw architecture on paper or whiteboard.
■ Algorithms	• LeetCode #146 — LRU Cache (classic design problem). • LeetCode #295 — Find Median from Data Stream.
■ Project Work	• No project work — full focus on system design.
✍■ Reflection	• Did I cover all the major components? • What trade-offs did I discuss? • What was missing from my design?

Day 82	Final Project — Start eCommerce API
■ Concepts	• Full project from scratch • Architecture-first • Product-level thinking
■ Theory & Understanding	• Your final project is a production-quality eCommerce REST API. This is the centrepiece of your portfolio. • Features: user registration/login (JWT), product catalog, cart, orders, payment simulation, admin panel. • Architecture: Clean Architecture. CQRS with MediatR. EF Core. Redis cache. RabbitMQ events. Docker.
■ YouTube Videos	• No video — design day.
■ Coding Practice	• Write the full requirements document. • Draw the domain model and entity relationships. • Design the API contract (all endpoints). • Create the solution structure.
■ Algorithms	• LeetCode #23 — Merge K Sorted Lists. • LeetCode #297 — Serialize and Deserialize Binary Tree.
■ Project Work	• eCommerceApi: project structure created, domain designed, EF Core configured, first migration.
✍■ Reflection	• Is the domain design correct? Are there missing entities? • What is the most complex feature to implement? • What will I use from everything I've learned?

Day 83	eCommerce API — Core Features
■ Concepts	• Putting it all together • Real-world complexity
■ Theory & Understanding	• Two days of pure building. Apply every pattern you've learned.
■ YouTube Videos	• No video.
■ Coding Practice	• Implement: Product catalog (CRUD, categories, pagination, search). • Implement: User registration and login (JWT, hashed passwords, roles). • Implement: Shopping cart (add, remove, update quantity).
■ Algorithms	• LeetCode #56 — Merge Intervals. • LeetCode #435 — Non-overlapping Intervals.
■ Project Work	• eCommerceApi: catalog + auth + cart all working and tested.
✍■ Reflection	• What patterns did I apply today? • What technical decision did I make and why? • What will I tackle tomorrow?

Day 84	eCommerce API — Orders & Admin
■ Concepts	• Order lifecycle • Admin operations • Advanced patterns
■ Theory & Understanding	• Orders are the most complex feature — they touch users, products, stock, payments, and events.
■ YouTube Videos	• No video.
■ Coding Practice	• Implement: Place order (transaction: decrement stock + create order + publish event). • Implement: Order history, order status updates. • Implement: Admin dashboard endpoints (stats, manage orders, manage products). • Add: Redis caching for product catalog. • Add: RabbitMQ event on order placement.
■ Algorithms	• LeetCode #91 — Decode Ways. • LeetCode #139 — Word Break.
■ Project Work	• eCommerceApi: all core features complete. Tests written. Docker container working.
✍■ Reflection	• How does the order flow work end-to-end? • What would break if two users ordered the last item simultaneously? How would you fix it? • Is the code clean enough to show to a senior engineer?

Day 85	Final Portfolio Polish
■ Concepts	• Code quality audit • Documentation • Deployment readiness
■ Theory & Understanding	• A polished portfolio is the difference between a callback and silence. • For each project: clean code, passing tests, CI badge, Swagger docs, comprehensive README, architecture diagram.
■ YouTube Videos	• No video.
■ Coding Practice	• Review ALL projects for: naming, dead code, commented-out code, consistent style. • Ensure all tests pass. CI is green for all repos.
■ Algorithms	• LeetCode #1 — Two Sum (5-min solve — confirm mastery). • LeetCode #206 — Reverse Linked List (3-min solve). • LeetCode #104 — Max Depth Binary Tree (5-min solve).
■ Project Work	• All 3 projects: TaskApi, BookstoreApi, eCommerceApi — polished and on GitHub.
✍■ Reflection	• If a recruiter looked at my GitHub right now, what would they think? • What is the narrative I want to tell about my growth over 85 days? • Am I ready for real interviews?

Day 86	Mock Interview — Full Technical Screen Simulation
■ Concepts	• Full 60-minute technical interview simulation
■ Theory & Understanding	• Format: 5 min introduction, 45 min coding problem, 10 min questions. • Treat this as real. No interruptions. Timer running.
■ YouTube Videos	• No video.
■ Coding Practice	• Simulate: one medium-hard LeetCode problem, timed, think out loud. • Then: 10 minutes of backend questions (ask yourself and answer).
■ Algorithms	• One randomly selected medium-hard problem.
■ Project Work	• Prepare 3 technical questions you would ask the interviewer.
✍■ Reflection	• Did I panic at any point? • Was my communication clear? • What do I need to work on in the final 4 days?

Day 87	Backend Interview Q&A; Drills
■ Concepts	• Common backend interview questions • Deep technical answers • Clear communication
■ Theory & Understanding	• Today: verbal drills. Answer each question out loud as if to an interviewer. • Questions cover: C#, ASP.NET Core, databases, algorithms, system design basics.
■ YouTube Videos	• No video.
■ Coding Practice	• Solve 3 easy LeetCode problems quickly — maintain momentum.
■ Algorithms	• LeetCode #20 — Valid Parentheses (3-min). • LeetCode #21 — Merge Two Sorted Lists (5-min). • LeetCode #121 — Best Time to Buy and Sell Stock (5-min).
■ Project Work	• Practice answering out loud: 'Explain SOLID', 'How does JWT authentication work?', 'What is async/await?', 'What is a database transaction?', 'Explain REST vs GraphQL', 'What is the repository pattern?', 'How would you design a URL shortener?', 'Explain Big-O notation'.
✍■ Reflection	• Which technical concept is hardest for me to explain clearly? • Am I concise? Or do I ramble? • What 2 topics need one final review before Day 90?

Day 88	Targeted Weak Area Review
■ Concepts	• Based on your reflection: attack your weakest areas
■ Theory & Understanding	• Look at all your reflection notes from the past 87 days. Find the recurring weak spots. Today is for them.
■ YouTube Videos	• Watch one video specifically targeted at your weakest area.
■ Coding Practice	• Write the algorithms or code patterns you feel weakest about — from scratch, no references.
■ Algorithms	• Revisit 2-3 problems you struggled with earlier in the plan. • Solve them again — have you improved?
■ Project Work	• Final README review. Add a 'What I Learned' section to each project README.
✍■ Reflection	• What has improved the most over 88 days? • What do I still find hard? • What is my mindset going into interviews?

Day 89	Final Mock Interview — Complete Simulation
■ Concepts	• Full interview day simulation
■ Theory & Understanding	• Today: simulate an entire interview day. Morning: code. Afternoon: system design. Evening: reflect. • No help. Timer running. Think out loud.
■ YouTube Videos	• No video.
■ Coding Practice	• 60 minutes: coding interview simulation.
■ Algorithms	• 45-minute timed coding problem — random medium.
■ Project Work	• 30-minute system design: Design a Rate Limiter.
✍■ Reflection	• How was my energy and focus throughout the day? • What went well compared to the first mock interview on Day 79? • Am I job-ready?

Day 90	Day 90 — You Made It. Now Go Build.
■ Concepts	• Reflection on the journey • Mindset for the job search • What comes next
■ Theory & Understanding	• You started as a weak CS graduate who struggled to start coding. You end as someone who has: • Built 3 production-quality APIs. Solved 100+ algorithmic problems. Learned async, auth, Docker, CI/CD, caching, system design. • The fundamentals are solid. The habits are formed. The portfolio is real. • Job searching is a skill itself. Apply persistently. Every rejection is data. • The most important habit: keep building. Keep writing code every day.
■ YouTube Videos	• No video. You've earned the silence.
■ Coding Practice	• Write: one page of what you know now that you didn't know 90 days ago. • Solve ONE problem you couldn't solve on Day 1.
■ Algorithms	• Choose any hard problem. Give it your best. Measure how far you've come.
■ Project Work	• Final commit: tag all three repos with v1.0.0. • Write a LinkedIn post about your 90-day journey (optional but powerful). • Start applying. You are ready.
✍■ Reflection	• What am I most proud of from the past 90 days? • What habit will I keep doing after this plan ends? • What kind of engineer do I want to be in one year?

QUICK REFERENCE — DATA STRUCTURE BIG-O

Structure	Access	Search	Insert	Delete	Notes
Array	O(1)	O(n)	O(n)	O(n)	Fixed size
Dynamic Array (List)	O(1)	O(n)	O(1) amort.	O(n)	Most common
Linked List	O(n)	O(n)	O(1) head	O(1) head	Good for queues
Stack	O(n)	O(n)	O(1)	O(1)	LIFO
Queue	O(n)	O(n)	O(1)	O(1)	FIFO
Hash Map	—	O(1) avg	O(1) avg	O(1) avg	Key-value
BST (balanced)	—	O(log n)	O(log n)	O(log n)	Red-Black
Heap (Min/Max)	O(1) min	O(n)	O(log n)	O(log n)	Priority queue
Graph (Adj List)	—	O(V+E)	O(1)	O(V+E)	BFS/DFS

ALGORITHM PATTERN CHEAT SHEET

Pattern	When to use	Key data structure	Complexity
Two Pointers	Sorted array, pairs, palindromes	Array indices	O(n)
Sliding Window	Subarray/substring problems	Deque or Hash	O(n)
Hash Map	Frequency, fast lookup, grouping	Dictionary	O(n)
Stack	Matching brackets, monotonic problems	Stack<T>	O(n)
BFS	Shortest path, level-by-level	Queue<T>	O(V+E)
DFS / Backtracking	All paths, combinations, permutations	Stack/Recursion	O(b^d)
Binary Search	Search in sorted space	lo/hi/mid pointers	O(log n)
Dynamic Programming	Overlapping subproblems, max/min/count	1D or 2D array	Varies
Greedy	Local optimal = global optimal	Priority Queue	Often O(n log n)
Topological Sort	Dependency ordering, cycle detection	Queue + in-degree	O(V+E)

You made it. Now go build something real. — Your Mentor